



Initiation à la méthodologie DevOps, Gitlab CI/CD et conteneurisation Docker

Partie 1 : Architecture & Fondations

Accueil et Présentation

Qui suis-je ?

Rapide présentation : parcours, ce que je fais au quotidien avec le DevOps et Docker, l'objectif de ces 5 jours avec vous.

Et vous ?

- Qu'avez-vous déjà fait avec Docker ?
- Qu'avez-vous fait en termes de mises en production ? Quels ont été vos pires soucis/bugs en production ?

Sondage rapide : qui a déjà mis quelque chose en production ? Qui a déjà utilisé Docker au quotidien ?

Objectifs de la formation

Voilà une vue d'ensemble des notions que l'on va apprendre ensemble.

1. **Maîtriser l'architecture DevOps et plus particulièrement Docker** : comprendre comment les pièces s'assemblent
 2. **Être opérationnel** : repartir avec des compétences directement applicables
-

Notre projet pour ce cours - Todo API

Voilà le projet fil rouge que nous allons construire :

En 5 jours, on dockerise, on met en pipeline et on déploie notre propre API de gestion de tâches (Todo API). Un projet volontairement simple côté code : l'objet du cours n'est pas la complexité applicative, mais de vivre toute la chaîne DevOps dessus, de A à Z, jusqu'à une mise en prod surveillée.

Chaque jour, on ajoutera une couche :

- **J1** : l'API Node.js tourne dans une stack Docker complète (Dockerfile, networks, volumes, docker-compose, image publiée)
- **J2** : on approfondit Docker (debug, optimisation d'images, healthchecks) et on ouvre sur l'orchestration
- **J3** : théorie CI/CD et DevOps, premier pipeline simple pour s'échauffer

- **J4** : pipeline GitLab-CI complet sur notre Todo API (tests, build, push, secrets) ; QCM
- Dans la semaine : déploiement sur Kubernetes, monitoring, rédaction d'une procédure de déploiement
- **J5** : projets de groupe démo finale du projet en groupe

Le jour 5, chaque groupe démontre son pipeline de bout en bout : un `git push`, et l'app part en prod sur Kubernetes, surveillée.

Comment on travaille

50/50. Moitié théorie, moitié pratique, et les deux sont entrelacés. Pas deux heures de slides suivies de deux heures de TP. On alterne.

Participation active : "Coupez"-moi la parole 🙋, pas de monologue d'enseignant, le but est que l'on discute et construise ensemble !

Découpage des chapitres : chaque section commence par **L'essentiel**, ce que tout le monde repart en maîtrisant. Puis on pousse plus loin, avec de la matière pour ceux qui avancent vite. On ajuste le curseur ensemble, en direct, selon le rythme du groupe.

Le matin on suit, l'après-midi on construit. Les quatre premiers chapitres se font ensemble, au même rythme. À partir du projet fil rouge, chacun avance sur son propre dépôt : les notions sont expliquées sur des exemples neutres, et les transposer sur son projet fait partie du travail. Personne n'est censé boucler les dix chapitres, ils sont calibrés pour ça.

Planning de la Journée

1. **Introduction DevOps et Architecture** : les 3 piliers du DevOps, le cycle DevOps, l'architecture type
2. **Les containers Linux sous le capot** : namespaces, cgroups, et un conteneur fabriqué à la main sans Docker
3. **Installation des Outils** : Docker Desktop, Node.js, Git, VS Code, compte de registry
4. **Architecture Docker et concepts de base** : daemon, client, registry, les commandes qui font tourner tout ça, et pour finir la construction de notre première image
5. **Le projet fil rouge Todo API (après-midi)** : le socle, une API de gestion de tâches qui répond, puis **Écrire un Dockerfile de production** : layers, cache, multi-stage, utilisateur non-root
6. **Networks et Volumes** : faire communiquer les conteneurs, faire survivre les données
7. **Docker Compose et la configuration** : toute la stack dans un fichier, configuration par variables d'environnement
8. **Un second service, en Python** : une stack polyglotte branchée sur la même base
9. **Publier l'image et redéployer** : le registry, et une stack qui démarre sans code source
10. **Mesurer et optimiser** : taille d'image, nombre de couches, temps de build à froid et à chaud

Vue d'ensemble - Points abordés dans cette Partie

Aperçu rapide de tout ce qui est vu aujourd'hui

Théories

- **Concepts DevOps fondamentaux**

- Définition du DevOps (collaboration Dev + Ops)
- Les 3 piliers : Culture, Automation, Measurement
- Le cycle DevOps en 8 étapes (Plan, Code, Build, Test, Release, Deploy, Operate, Monitor)
- Architecture DevOps type avec feedback loop

- **La nature d'un container Linux**

- Un conteneur est un processus, pas une machine
- Les namespaces (pid, net, mnt, uts, ipc, user) : ce que le processus a le droit de voir
- Les cgroups : ce que le processus a le droit de consommer
- Le système de fichiers en couches (chroot, overlayfs)
- Conteneurs vs machines virtuelles, et les cas où la VM reste le bon choix

- **Introduction à Docker**

- Problématique du "ça marche sur ma machine"
- Avantages de Docker (rapidité, multiplateforme, isolation, déploiement facilité)
- Architecture client / daemon / registry
- Docker vs Kubernetes vs Docker Swarm

- **Terminologie Docker essentielle**

- Dockerfile : script de construction d'image
- Image : snapshot immuable et exécutable
- Container : instance en cours d'exécution
- Registry, Volume, Network : les trois autres briques du vocabulaire
- La relation entre tous ces concepts

- **Le Dockerfile de production**

- Les instructions qui comptent, et CMD vs ENTRYPOINT
- Layers et cache de build : pourquoi l'ordre des lignes change tout
- `.dockerignore`, utilisateur non-root, images de base épinglées
- Multi-stage builds et réduction de la taille d'image
- Spécificités Node.js et spécificités Python

- **Les Networks Docker**

- Pourquoi deux conteneurs ne se voient pas par défaut
- Les drivers bridge, host, none
- La résolution DNS par nom de service sur un network custom
- Isolation entre networks

- **Les Volumes Docker**

- Problématique de la persistance des données
- Les 3 types de montage (volumes, bind mounts, tmpfs)

- Différence entre volumes nommés et anonymes
- Syntaxes -v vs --mount
- Partage de données entre conteneurs
- **Docker Compose et configuration**
 - Orchestration de services multi-conteneurs
 - Syntaxe du fichier docker-compose.yml
 - Services, networks, volumes
 - Commandes principales (build, up, down)
 - Gestion des dépendances entre services (depends_on) et ses limites
 - Configuration par variables d'environnement, `.env` et secrets
 - Ouverture sur les bonnes pratiques de conteneurisation en production
- **Publication et redéploiement**
 - Push d'images vers un registry public ou privé
 - Tags d'images et versionnement sémantique
 - Pull et redéploiement depuis le registry
- **Métriques et optimisation d'images**
 - Taille d'une image, nombre de couches, poids de chaque couche
 - Build à froid contre build à chaud : ce que le cache fait gagner
 - Temps écoulé entre le lancement d'un conteneur et sa première réponse
 - Ce qu'une optimisation fait gagner, et le cas où elle fait régresser



Mises en pratique

- **Fabriquer un container à la main, sans Docker**
 - Récupération d'un rootfs minimal
 - Isolation du système de fichiers avec chroot
 - Isolation des processus, du réseau et du hostname avec unshare
 - Limitation de la mémoire avec un cgroup, et le processus qui se fait tuer
- **Découverte des commandes de base**
 - `docker pull` : récupérer une image depuis un registry
 - `docker create` puis `docker start`, et pourquoi `docker run` fait les deux
 - `docker ps`, `docker ps -a`, `docker logs`, `docker exec`
 - `docker stop`, `docker rm`, `docker rmi` et la différence entre les trois
 - Cycle de vie complet d'un conteneur
- **Installation et configuration de l'environnement**
 - Installation de Docker Desktop
 - Vérification avec `docker --version` et hello-world
 - Configuration de VS Code avec extensions Docker
 - Configuration Git (user.name, user.email)

- **Manipulation via terminal**
 - Commandes bash essentielles (pwd, cd, ls, mkdir, touch, cat)
 - Navigation dans le système de fichiers
 - Exécution de commandes dans les conteneurs (docker exec)
- **Navigation dans Docker Desktop**
 - Exploration de l'interface (Containers, Images, Volumes)
 - Visualisation des logs graphiquement
 - Gestion des conteneurs depuis l'interface
- **Premier Dockerfile et image**
 - Création d'un Dockerfile pour une app Node.js
 - Build d'une image avec docker build -t
 - Compréhension des layers lors du build
 - Exposition de ports avec -p (mapping host:container)
- **Exercice Networks**
 - Création d'un network custom
 - Deux conteneurs qui se pinguent par leur nom
 - Inspection du network et de ses IPs
 - Démonstration de l'isolation entre deux networks
- **Exercice Volumes**
 - Création d'un volume nommé avec docker volume create
 - Montage du volume sur plusieurs conteneurs
 - Écriture et lecture de données partagées
 - Test de persistance après suppression des conteneurs
 - Inspection avec docker volume inspect
- **Stack complète en docker-compose**
 - Configuration d'une stack Node.js + base de données
 - Configuration des ports, des networks et des environnements
 - Utilisation de docker compose up/down/build
- **Todo API : le projet de l'après-midi**
 - Chapitre 5 : API CRUD dockerisée en Node.js, avec un Dockerfile de production optimisé
 - Chapitre 6 : Ajout d'une base PostgreSQL, volumes de persistance, communication via network
 - Chapitre 7 : Orchestration de la stack en docker-compose, configuration entièrement par variables d'environnement
 - Chapitre 8 : Service en Python branché sur la même stack (calcul, agrégation ou transformation de données)
 - Chapitre 9 : Publication des images sur un registry, puis redéploiement depuis le registry

- Chapitre 10 : Mesure de la taille d'image, optimisation des layers, inspection des temps de build et de startup

1 - Introduction DevOps et Architecture

L'essentiel

Qu'est-ce que le DevOps ?

Le DevOps, c'est faire collaborer les développeurs (Dev) et les opérationnels (Ops) pour livrer du code plus vite et plus sûrement.

Quel est le problème quand Dev et Ops ne se parlent pas ?

1. Définir et configurer une architecture DevOps

Une architecture DevOps est un ensemble d'outils et de pratiques qui permettent aux équipes de développement et d'opérations de collaborer efficacement.

Avec Docker, on peut créer des conteneurs pour nos applications, ce qui facilite leur déploiement et leur gestion. Nous creuserons cela un peu plus tard.

2. Automatisation du cycle de vie d'une application

Grâce au CI / CD (Intégration Continue / Déploiement Continu) sur GitHub, vous pouvez automatiser le processus de test et de déploiement de votre application.

Cela signifie que chaque fois que vous apportez des modifications, des tests unitaires sont exécutés automatiquement pour s'assurer que tout fonctionne avant de déployer les changements !

3. Outils, techniques

Nous les observerons au fil de ce cours.

4. Supervision et Performance des Applications

Il est essentiel de surveiller nos applications et notre infrastructure pour garantir leur disponibilité, leurs performances, éviter les imprévus.

Des outils de monitoring peuvent nous alerter en cas de problème.

Sur le dernier projet nous pourrons expérimenter cela.

Le terme n'est pas sorti de nulle part : il vient de la [DevOpsDays](#) de Gand en 2009, une conférence montée par Patrick Debois pour réunir deux mondes qui s'ignoraient. Depuis, c'est devenu un mot-valise que tout le monde met sur sa fiche de poste, ce qui rend d'autant plus utile de savoir ce qu'il y a vraiment dedans.

Les 3 piliers du DevOps

1. Culture

- Collaboration entre équipes
- Responsabilité partagée
- Feedback continu

2. Automation

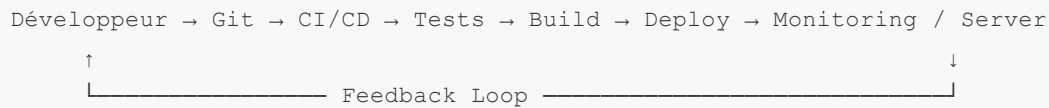
- Tests automatisés
- Déploiement automatique

- Infrastructure as Code

3. Measurement

- Métriques de performance
- Monitoring applicatif
- Alerting intelligent

Architecture DevOps type



Ce qui compte dans ce schéma, c'est la flèche du bas. Sans elle, on a juste une chaîne de montage : le code part, et personne ne sait ce qu'il devient. La boucle de retour, c'est ce qui transforme une suite d'outils en méthode de travail.

Le cycle DevOps

1. **Plan** : Définir les features
2. **Code** : Développer
3. **Build** : Compiler/packager
4. **Test** : Valider automatiquement
5. **Release** : Préparer la mise en production
6. **Deploy** : Mettre en production
7. **Operate** : Maintenir en conditions opérationnelles
8. **Monitor** : Observer et mesurer

Sur ces huit étapes, notre journée d'aujourd'hui couvre surtout **Build** et **Deploy** : on apprend à emballer une application de façon reproductible. Le reste de la semaine remontera la chaîne, jusqu'à ce que le cycle tourne tout seul.

Pour les curieux

Comment on sait qu'une équipe "fait bien" du DevOps ? À quoi ça se mesure, concrètement ?

Il existe une réponse chiffrée, et elle est publique. Le programme de recherche [DORA](#) (DevOps Research and Assessment, aujourd'hui hébergé par Google Cloud) interroge chaque année des dizaines de milliers d'équipes et publie un rapport, l'[Accelerate State of DevOps Report](#). Sa conclusion la plus reprise tient en quatre métriques.

Les 4 métriques DORA ? Quatre indicateurs qui, mis ensemble, prédisent assez bien la performance d'une équipe de livraison logicielle.

- **Deployment frequency** : à quelle fréquence on met en production. Les équipes les plus performantes déploient plusieurs fois par jour, les moins performantes une fois par mois ou moins.
- **Lead time for changes** : le temps entre un commit et sa mise en production. On parle de moins d'une heure d'un côté, de plusieurs mois de l'autre.
- **Change failure rate** : la proportion de déploiements qui cassent quelque chose.

- **Time to restore service** : le temps qu'il faut pour se remettre debout quand ça casse.

Le point contre-intuitif, et c'est celui qui vaut le détour : les deux premières métriques (la vitesse) et les deux dernières (la stabilité) ne s'opposent pas. On aurait envie de croire qu'aller vite fait casser plus souvent. Les données disent l'inverse, les équipes qui déploient le plus souvent sont aussi celles qui cassent le moins. La raison est mécanique : de petits déploiements fréquents sont plus faciles à tester, à comprendre et à annuler qu'un gros déploiement trimestriel.

Deux autres notions vous suivront toute votre carrière, autant les poser maintenant.

Infrastructure as Code (IaC) ? Décrire ses serveurs, ses réseaux et ses services dans des fichiers versionnés au lieu de les configurer à la main. Un `Dockerfile` et un `docker-compose.yml`, c'est déjà de l'Infrastructure as Code.

SRE ? Site Reliability Engineering, l'approche formalisée par Google ([le livre est en accès libre](#)) : traiter l'exploitation comme un problème de logiciel, avec des objectifs de fiabilité chiffrés (les SLO) et un budget d'erreur assumé.

Côté carrière, c'est un domaine où les certifications ont encore du poids : la [Certified Kubernetes Administrator](#) de la CNCF, ou les certifications cloud (AWS, Azure, GCP) sont régulièrement demandées sur les offres DevOps et pèsent réellement sur les grilles de salaire. On reparlera de Kubernetes en fin de semaine.

2 - Les containers Linux : ce qu'il y a sous le capot

L'essentiel

Quand vous tapez `docker run nginx`, qu'est-ce qui se passe vraiment sur votre machine ? Une mini machine virtuelle qui démarre quelque part ? Un logiciel qui s'installe comme n'importe quel programme ?

Ni l'un ni l'autre. Ce qui démarre, c'est un processus Linux tout à fait normal, visible depuis votre machine hôte, qui tourne sur le même noyau (kernel) que le reste de votre système. La seule différence avec un processus classique, c'est qu'on lui a mis des œillères : il croit être seul sur la machine, avoir son propre système de fichiers, son propre réseau, ses propres utilisateurs. Cette illusion, ce sont trois mécanismes du noyau Linux qui la fabriquent : les namespaces, les cgroups, et un système de fichiers en couches. Avant de reparler de Docker, on va comprendre ces trois briques, parce que tout le reste du cours (isolation, limites de ressources, sécurité) en découle directement.

Les namespaces : l'illusion d'être seul

Un conteneur n'est isolé que parce que le noyau lui montre une vue partielle et cloisonnée de la réalité. Ce mécanisme s'appelle les [namespaces](#) (espaces de noms) : une fonctionnalité du noyau Linux qui permet de faire croire à un groupe de processus qu'ils sont seuls à voir une ressource du système (une liste de processus, une interface réseau, un point de montage...), alors que d'autres processus, ailleurs sur la même machine, en voient une version complètement différente.

Namespace ? Un mécanisme du noyau Linux qui cloisonne la vue d'une ressource système (processus, réseau, points de montage...) pour un groupe de processus donné, sans dupliquer le matériel ni le noyau.

Il en existe plusieurs sortes, chacune cloisonnant une ressource différente :

Instant Notions : les namespaces qu'on va manipuler aujourd'hui

- **pid** : cloisonne la liste des processus visibles. Dans son propre namespace pid, un processus peut se croire PID 1, même s'il est PID 48213 côté hôte.
- **net** : cloisonne les interfaces réseau, les routes, les ports. Un conteneur peut écouter sur le port 80 sans entrer en conflit avec un serveur qui écoute déjà sur le port 80 côté hôte.
- **mnt** : cloisonne les points de montage, donc la vue du système de fichiers. C'est ce qui permet à un conteneur de voir sa propre racine `/` sans voir celle de l'hôte.
- **uts** : cloisonne le hostname et le nom de domaine. On peut changer le hostname à l'intérieur sans toucher à celui de la machine hôte.
- **ipc** : cloisonne les mécanismes de communication entre processus (mémoire partagée, files de messages).
- **user** : cloisonne la correspondance entre utilisateurs. Un processus peut être "root" à l'intérieur de son namespace tout en étant un utilisateur non privilégié côté hôte. C'est la brique centrale des conteneurs rootless, qu'on retrouve un peu plus loin.

Le point important à retenir : les namespaces ne créent rien de nouveau, ils **cachent**. Le noyau reste unique, partagé par tous les conteneurs et par l'hôte. C'est la différence fondamentale avec une machine virtuelle, on y revient dans un instant.

Les cgroups : la laisse sur les ressources

Isoler la vue d'un processus, c'est bien, mais rien n'empêche encore ce processus de manger toute la RAM ou tout le CPU de la machine. C'est le rôle des [cgroups](#) (control groups) : une fonctionnalité du noyau qui permet de regrouper des processus et de leur appliquer des limites et une comptabilité sur leur consommation de ressources (CPU, mémoire, IO disque, réseau).

cgroup ? Un mécanisme du noyau Linux qui limite, priorise et mesure la consommation de ressources (CPU, RAM, IO) d'un groupe de processus.

La version actuelle, cgroup v2, est documentée dans la [documentation officielle du noyau Linux](#).

Concrètement, ça se manipule comme un système de fichiers virtuel monté sous

`/sys/fs/cgroup/` : on crée un dossier, on écrit une valeur dans un fichier comme `memory.max`, et le noyau applique la limite à tous les processus rattachés à ce cgroup. Si un processus dépasse la limite mémoire fixée, le noyau ne négocie pas : il le tue (c'est ce qu'on appelle un OOM-kill, Out Of Memory kill). On va le déclencher nous-mêmes dans quelques minutes.

Namespaces et cgroups répondent à deux questions différentes et complémentaires : *qu'est-ce que ce processus a le droit de voir ?* (namespaces) et *combien de ressources a-t-il le droit de consommer ?* (cgroups).

Le système de fichiers en couches

Reste un dernier problème : d'où vient le `/` que voit un conteneur une fois isolé par son namespace mnt ? Il faut bien un dossier réel, quelque part sur le disque de l'hôte, qui serve de racine. Deux mécanismes s'enchaînent historiquement pour construire ça.

Le plus ancien, c'est le [chroot](#) : une commande qui change la racine `/` apparente d'un processus vers un sous-dossier arbitraire. Un processus chrooté dans `/mon-dossier` croit que `/mon-dossier` est la racine du système, et ne peut pas remonter au-dessus. C'est simple, ça existe depuis des décennies, mais ce n'est **pas** de l'isolation de sécurité : chroot ne touche ni aux

processus visibles, ni au réseau, ni aux utilisateurs. On le vérifie nous-mêmes juste en dessous.

pivot_root ? Un appel système plus robuste que `chroot`, utilisé en interne par les outils de conteneurisation modernes (dont Docker) pour basculer complètement la racine du système de fichiers d'un processus, en rendant l'ancienne racine inaccessible plutôt que simplement masquée.

Enfin, pour éviter de dupliquer des gigaoctets de fichiers identiques à chaque image, Docker construit ce dossier racine en empilant des couches en lecture seule les unes sur les autres, avec [overlayfs](#) comme pilote de stockage par défaut. Chaque instruction d'un Dockerfile ajoute une couche ; au démarrage d'un conteneur, une dernière couche en lecture-écriture vient se poser au sommet de la pile. On reparlera d'overlayfs plus tard dans le programme quand on regardera le détail des layers d'une image.

Conteneur vs machine virtuelle : le scénario qui tranche

Imaginez : on doit lancer 50 instances d'une même application pour un test de charge, un vendredi après-midi, avant une mise en prod le lundi.

Avec des **machines virtuelles**, chaque instance embarque son propre noyau complet, démarre un système d'exploitation entier (BIOS virtuel, init, services système) avant même de lancer l'application. Comptez 30 secondes à 2 minutes de démarrage par VM, et plusieurs gigaoctets de disque chacune, puisqu'un noyau complet et un OS complet doivent être dupliqués à chaque fois. 50 VMs, c'est potentiellement plusieurs centaines de Go et un bon quart d'heure avant d'avoir tout le monde debout.

Avec des **conteneurs**, on saute la case démarrage d'OS : le noyau est déjà là, partagé, déjà démarré, c'est celui de la machine hôte. Un conteneur, c'est juste un processus qu'on lance avec des œillères (namespaces) et une laisse (cgroups) autour. Démarrage en moins d'une seconde, poids en dizaines ou centaines de Mo par image (souvent bien moins avec une image de base légère). 50 conteneurs démarrent en quelques secondes.

Ce gain de poids et de vitesse a un prix : la frontière d'isolation. Une VM est isolée au niveau matériel par un hyperviseur, avec son propre noyau : une faille dans le noyau d'une VM ne donne, en théorie, pas directement accès à l'hôte ni aux autres VMs. Un conteneur partage le même noyau que l'hôte et que tous les autres conteneurs de la machine : une faille noyau exploitée depuis un conteneur peut, dans le pire cas, permettre d'en sortir. C'est pour ça que la VM reste le bon choix quand on héberge du code de plusieurs clients qui ne se font pas confiance entre eux (isolation multi-tenant forte, hébergeurs cloud), quand on doit faire tourner un noyau différent de celui de l'hôte (un workload Windows sur un hôte Linux, par exemple), ou quand une exigence de conformité impose une frontière matérielle et pas seulement logicielle.

Instant Notions : conteneur vs VM en un coup d'œil

- **Démarrage** : conteneur, moins d'une seconde. VM, 30 secondes à 2 minutes.
- **Poids** : conteneur, dizaines à centaines de Mo. VM, plusieurs Go (OS + noyau complets).
- **Isolation** : conteneur, cloisonnement logiciel sur un noyau partagé. VM, cloisonnement matériel via un hyperviseur, noyau dédié.
- **Bon choix VM** : multi-tenant non fiable, noyau différent requis, conformité exigeant une frontière matérielle.

Ce matin, on construit un conteneur à la main (sans Docker)

Avant de retoucher à Docker, on va fabriquer notre propre mini-conteneur avec les seuls outils du noyau Linux : pas de `docker`, pas de `Dockerfile`, juste `chroot`, `unshare` et les `cgroups`. L'objectif n'est pas de retenir ces commandes par cœur, mais de voir de vos propres yeux que Docker n'a rien de magique : c'est un empilement de ces mêmes briques, avec une jolie interface autour.

💡 Ces commandes sont des fonctionnalités du noyau **Linux** : elles ne fonctionnent pas nativement sur macOS ni Windows, puisque ces systèmes n'ont pas ce noyau. Si votre machine est un Mac ou un PC Windows, le plus simple est une VM Linux légère (WSL2 sous Windows fait très bien l'affaire, ou une VM Multipass/UTM). Sinon, on suit la démo à l'écran et on la rejoue juste après la section suivante, une fois Docker installé, dans un conteneur "privilégié" lancé avec `docker run -it --rm --privileged alpine sh`. Le comble de l'exercice, oui : utiliser Docker comme bac à sable Linux pour recréer un conteneur sans Docker.

Étape 1 : se procurer un système de fichiers racine

On a besoin d'un dossier qui contienne un mini système Linux complet (un `/bin`, un `/etc`, un `/lib`...) pour servir de racine à notre futur conteneur. Deux chemins classiques existent : [debootstrap](#), un outil qui installe un système Debian minimal dans un sous-dossier, ou plus simple et plus rapide pour l'exercice du jour, récupérer directement l'archive `alpine-minirootfs` publiée officiellement par [Alpine Linux](#), une distribution ultra-légère taillée pour les conteneurs.

```
# On crée un dossier de travail, avec un sous-dossier dédié au futur rootfs
mkdir -p ~/mini-container/rootfs
cd ~/mini-container

# On télécharge l'archive minirootfs d'Alpine (quelques Mo seulement, contre
# centaines de Mo pour une image Ubuntu ou Debian complète)
curl -L -o alpine-minirootfs.tar.gz \
  https://dl-cdn.alpinelinux.org/alpine/latest-stable/releases/x86_64/alpine-
minirootfs-3.20.3-x86_64.tar.gz

# On extrait l'archive DANS rootfs/ : ce dossier va devenir la racine "/" de
# notre conteneur
tar -xzf alpine-minirootfs.tar.gz -C rootfs
```

Devrait afficher, en tapant `ls rootfs` : une arborescence Linux classique, `bin`, `etc`, `home`, `lib`, `root`, `sbin`, `usr`, `var`... Ce dossier va littéralement devenir le `/` de notre mini-conteneur.

Étape 2 : chroot seul, et sa limite

On isole d'abord juste la vue du système de fichiers, avec `chroot` :

```
sudo chroot rootfs /bin/sh
```

Devrait afficher : un prompt `/ #` tout neuf. On tape `ls /`, on voit le contenu d'Alpine et non celui de la machine hôte : le système de fichiers est bien cloisonné.

Maintenant, le piège. On tape `ps aux` (ou `ps` si `ps aux` n'est pas reconnu selon la version de busybox) :

Devrait afficher : la liste complète des processus de la machine **hôte**, pas seulement ceux du chroot. Rien à voir avec un conteneur isolé : `chroot` ne touche ni au namespace pid, ni au réseau, ni aux utilisateurs, seulement à la vue du système de fichiers. C'est exactement la limite qu'on annonçait plus haut.

On sort avec `exit`.

Étape 3 : chroot + unshare, la vraie isolation

On combine maintenant `chroot` avec [unshare](#), la commande qui crée de nouveaux namespaces pour le processus qu'elle lance :

```
sudo unshare --pid --fork --mount-proc --uts --net -- chroot rootfs /bin/sh
```

- `--pid` : nouveau namespace pid, notre shell devient PID 1 dans son propre monde
- `--fork` : indispensable avec `--pid`, pour que le processus qui obtient le nouveau namespace soit bien celui qu'on lance
- `--mount-proc` : remonte un `/proc` propre au nouveau namespace pid (sinon `ps` continuerait de lire l'ancien `/proc` de l'hôte)
- `--uts` : nouveau namespace pour le hostname
- `--net` : nouveau namespace réseau, vide au départ

Une fois dans le shell obtenu, on teste :

```
hostname mon-conteneur-a-la-main  
ps aux
```

Devrait afficher : pour `hostname`, aucune erreur, et si on ouvre un second terminal côté hôte pour taper `hostname`, on voit que le hostname de la machine hôte n'a pas bougé (namespace uts cloisonné). Pour `ps aux`, cette fois une poignée de processus seulement, essentiellement `/bin/sh` en PID 1 : c'est la preuve concrète du cloisonnement pid.

On peut aussi taper `ip addr` (busybox l'embarque) : **devrait afficher** uniquement l'interface `lo` (loopback), aucune interface réseau vers l'extérieur. Le namespace net est bien vide, notre mini-conteneur n'a aucune connectivité, exactement comme un vrai conteneur Docker sans réseau attaché.

On sort avec `exit`.

Étape 4 : la laisse cgroup, et la démo qui tue

Dernière brique : on limite la mémoire disponible avec un cgroup, et on regarde le noyau appliquer la sanction.

```
# On crée un cgroup dédié à l'exercice (un simple sous-dossier sous le point de
montage cgroup v2)
sudo mkdir /sys/fs/cgroup/demo-mini-container

# On fixe une limite mémoire volontairement basse : 20 Mo
echo 20M | sudo tee /sys/fs/cgroup/demo-mini-container/memory.max

# On lance un shell isolé, on l'attache au cgroup en écrivant son PID dans
cgroup.procs,
# puis on tente d'allouer 200 Mo d'un coup, dix fois la limite fixée
sudo unshare --fork --pid --mount-proc -- sh -c '
    echo $$ | sudo tee /sys/fs/cgroup/demo-mini-container/cgroup.procs
    python3 -c "a = bytearray(200 * 1024 * 1024)"
    '
```

Devrait afficher : le message `Killed`, le processus Python meurt avant même de terminer son allocation. En tapant `dmesg | tail` côté hôte juste après, on voit une ligne mentionnant `Memory cgroup out of memory` ou `oom-kill` : c'est le noyau qui a appliqué la limite fixée dans `memory.max`, pas une erreur applicative. C'est très exactement ce que Docker fait derrière un `docker run --memory 20m`.

Qu'est-ce qui se passerait si on avait fixé la limite à 500 Mo au lieu de 20 Mo ? Le processus tiendrait la charge, aucun kill : la limite n'est un problème que quand on la dépasse réellement, elle est totalement transparente en dessous.

On nettoie derrière nous :

```
sudo rmdir /sys/fs/cgroup/demo-mini-container
```

À ce stade, on a recréé à la main les trois ingrédients d'un conteneur : une vue de système de fichiers isolée, des processus et un réseau cloisonnés, une consommation de ressources bridée. Tout ce que Docker ajoute par-dessus, c'est de l'orchestration et de l'ergonomie autour de ces mêmes appels noyau.

Creusons

Ce que fait Docker avec les mêmes briques

Quand on tape `docker run`, Docker n'invente rien de plus que ce qu'on vient de faire à la main. La partie qui orchestre concrètement la création des namespaces, l'application des limites cgroups et le `pivot_root` vers le système de fichiers de l'image s'appelle `runc` : un outil en ligne de commande bas niveau, développé à l'origine sous le nom `libcontainer` chez Docker puis donné à la communauté, dont le rôle unique est de démarrer un conteneur conforme à une spécification standardisée.

Cette spécification, c'est l'[OCI Runtime Specification](#), portée par l'Open Container Initiative. Elle décrit, dans un format texte standardisé (`config.json`), quelles namespaces créer, quelles limites cgroups appliquer, quel `rootfs` monter. L'intérêt : n'importe quel outil qui sait produire ou lire ce format (Docker, mais aussi Podman, Kubernetes via containerd...) peut faire tourner le même conteneur, sans être lié à l'implémentation Docker.

OCI ? Open Container Initiative : une organisation qui définit des standards ouverts pour le format des images et des runtimes de conteneurs, afin qu'ils soient interchangeables entre outils.

Pourquoi `--privileged` est dangereux

On a utilisé `docker run --privileged` plus haut pour rejouer l'exercice sur une machine sans noyau Linux natif. Il faut bien comprendre ce que ce flag fait réellement : il désactive une bonne partie des restrictions de sécurité qu'on vient de voir. Un conteneur privilégié a accès à tous les périphériques de l'hôte, peut charger des modules noyau, et peut dans de nombreux cas remonter jusqu'au système de fichiers de l'hôte lui-même. Concrètement, un conteneur privilégié compromis, c'est une machine hôte compromise. C'est un flag pratique pour un exercice pédagogique en salle de classe, jamais acceptable pour un conteneur qui tourne en production sans raison impérieuse et documentée.

Voir dans un conteneur qui tourne déjà : `nsenter`

Autre outil bas niveau utile à connaître : [nsenter](#), qui permet d'entrer dans les namespaces d'un processus déjà en cours d'exécution, y compris ceux d'un conteneur Docker déjà lancé, sans passer par `docker exec`. C'est ce genre d'outil qu'on utilise pour du débogage bas niveau, par exemple inspecter les tables de routage réseau exactes d'un conteneur depuis l'hôte, ou diagnostiquer un conteneur dont le processus principal ne répond plus mais dont les namespaces existent toujours.

Les conteneurs rootless

On a mentionné le namespace user plus haut : c'est la brique qui permet à un processus d'être "root" à l'intérieur de son conteneur tout en étant un utilisateur non privilégié côté hôte. C'est le principe des conteneurs [rootless](#) : faire tourner le démon Docker lui-même, et les conteneurs qu'il lance, sans jamais avoir besoin de droits root sur la machine hôte. L'intérêt en sécurité est direct : si un attaquant s'échappe d'un conteneur rootless, il atterrit sur un utilisateur non privilégié de l'hôte, pas sur root. C'est de plus en plus la configuration recommandée en production, même si elle a quelques limitations (certains ports privilégiés sous 1024, certaines fonctionnalités réseau avancées).

3 - Installation des Outils

On sait maintenant ce qu'il y a vraiment sous un conteneur. À partir de là, tout le reste de la journée se passe les mains dans le terminal, donc prenons dix minutes pour que tout le monde ait exactement le même outillage sur sa machine. C'est le moment de régler les problèmes d'installation, pas dans deux heures au milieu du projet.

L'essentiel

Les prérequis

Sondage rapide : Qui a déjà installé Docker ? Node.js ? Git ? Quelle est votre stack principale / vos stacks acquises ?

Installation guidée

1. Docker Desktop (+ Docker terminal)

Vérification de la présence de Docker

```
# Vérification
docker --version
docker compose version

# Test rapide
docker run hello-world
```

Pour installer :

Aller sur le site officiel de Docker : <https://www.docker.com/get-started>

Télécharger selon votre OS.

Suivre les instructions d'installations spécifique à l'OS.

Docker sera présent en version Desktop et dans le terminal.

"Hello world ne marche pas chez moi"

Si certaines personnes ont un message contenant 401 unauthorized, c'est que la requête fonctionne, mais que l'accès est non autorisé. Il faut se connecter :

Sur terminal (Git Bash ou Powershell) :

```
docker login -u <mon_username>
```

2. Installez Docker sur VSCode

Cela permet d'avoir un support langage, pendant qu'on rédige nos fichiers docker. Pratique pour avoir les bonnes couleurs et de l'aide.

Nous pouvons également faire un lien vers des remote registeries.

3. Node.js (v18+) (normalement vous l'avez !)

```
# Via nvm (recommandé)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh |
bash
nvm install 18
nvm use 18

# Vérification
node --version
npm --version
```

4. Git & GitLab / Github

```
# Configuration
git config --global user.name "Votre Nom"
git config --global user.email "email@example.com"
```

5. VS Code + Extensions

- Docker
- GitLens
- ESLint
- REST Client

Un compte de registry, à créer maintenant

En fin de journée, on publiera nos images en ligne. Autant créer le compte tout de suite pendant qu'on est dans la phase installation : un compte gratuit sur [Docker Hub](https://hub.docker.com/), le registry public opéré par Docker, où l'on dépose et récupère des images.

Registry ? Un dépôt d'images Docker, l'équivalent de npm pour les paquets Node ou de PyPI pour les paquets Python.

Une fois le compte créé, on vérifie que la connexion passe depuis le terminal :

```
docker login
# Devrait afficher : Login Succeeded
```

Environnement de secours

S'il y a trop de problèmes pour l'installation locale, voici une alternative :

GitHub Codespaces comme backup :

1. Créer un compte GitHub si nécessaire
2. Utiliser un repo test, fait par vos soins
3. Lancer un Codespace

Instant Cheat Sheet ! => Terminal cheat sheet

Pour les personnes ayant besoin d'un rappel sur bash

Comment naviguer dans le système de fichiers ?

- `pwd` : savoir où on se trouve
- `cd chemin_du_dossier` : aller dans un dossier (et `cd ..` pour revenir au dossier parent)
- `ls`, ou `ls -l`, ou `ls -la` : lister les éléments

Commandes de manipulation de fichiers :

- `mkdir nom_de_dossier` : créer un dossier
- `touch nom_du_fichier.txt` : créer un fichier
- `cat nom_du_fichier` : afficher le contenu du fichier

Commandes très pratiques :

- ajouter `--help` à la suite de la commande sur laquelle on se pose des questions
- écrire `man le_nom_de_ma_commande` pour en savoir plus sur la commande (man veut dire manual)

On peut éditer nos fichiers directement dans le terminal ?

-> Oui, carrément : des outils d'éditeurs de texte comme [nano](#) et [vim](#) permettent de manipuler nos fichiers. Je conseille vim, que j'utilise beaucoup ! (mais c'est une question d'habitude)

On va plus loin

Docker Desktop, c'est quoi exactement ? Et pourquoi ça demande autant de RAM sur mon Mac ?

Bonne question, et la réponse explique pas mal de bizarreries de la journée. Le moteur Docker est un logiciel Linux : il s'appuie sur les namespaces et les cgroups du noyau Linux, qui n'existent tout simplement pas sur macOS ni sur Windows. Donc sur ces deux systèmes, Docker Desktop démarre discrètement une machine virtuelle Linux légère, et c'est elle qui fait tourner vos conteneurs. Le client `docker` de votre terminal parle à un daemon qui vit dans cette VM.

Trois conséquences très concrètes :

- Les montages de dossiers depuis votre machine vers le conteneur (les bind mounts, vus plus loin) traversent une frontière de système de fichiers, ce qui les rend nettement plus lents sur macOS et Windows que sur Linux. Si un `npm install` monté depuis l'hôte rame, c'est ça.
- La mémoire allouée à cette VM est réglable dans les préférences de Docker Desktop, et c'est le premier endroit où regarder quand un build échoue avec un message d'épuisement mémoire.
- Sur Windows, [WSL 2](#) (Windows Subsystem for Linux, une couche qui fait tourner un vrai noyau Linux dans Windows) sert de moteur. Travailler depuis un dossier WSL plutôt que depuis `C:\Users\...` change radicalement les performances.

Il existe aussi des alternatives à Docker Desktop, utiles à connaître parce que la licence Docker Desktop est payante pour les entreprises de plus de 250 personnes :

- [Podman](#) : un moteur compatible avec les mêmes commandes, sans daemon permanent et capable de tourner sans privilèges root.
- [Colima](#) : une VM Linux minimaliste pour macOS, pilotée en ligne de commande.
- [Rancher Desktop](#) : une alternative graphique complète, qui embarque en plus un Kubernetes local dont on reparlera en fin de semaine.

Ces outils exposent la même interface en ligne de commande, donc tout ce qu'on apprend aujourd'hui reste valable si vous basculez sur l'un d'eux.

4 - Architecture Docker et concepts de base

L'essentiel

On ne le nomme plus : [Fireship](#) ! Si vous voulez démystifier un sujet rapidement allez sur sa chaîne. Pour le cours du jour : [Docker in 100 Seconds](#)

De nos jours, le développement d'applications ne se limite pas à l'écriture de code. La multiplicité des langages, des cadres, des architectures et des interfaces discontinues entre les outils pour chaque étape du cycle de vie crée une énorme complexité.

👉 Docker simplifie et accélère votre flux de travail, tout en donnant aux développeurs la liberté d'innover en choisissant leurs outils, leurs piles d'applications et leurs environnements de déploiement pour chaque projet.

Les conteneurs sont une unité logicielle standardisée qui permet aux développeurs d'isoler leur application de son environnement, ce qui résout le problème du "ça marche sur ma machine".

Pourquoi docker ? -> [Why Docker | Docker](#)

Très bonne ressource à suivre : [Docker Simplified: A Hands-On Guide for Absolute Beginners](#)

Pourquoi utiliser Docker en tant que développeur ?

Avantages de Docker :

- Rapidité: Démarrage et arrêt rapides de l'application.
- Multiplateforme: Fonctionne sur n'importe quel système.
- Construction et destruction rapides des conteneurs.
- Configuration facile: Évite les problèmes d'installation manuelle des dépendances.
- Environnements isolés: Maintient la propreté de l'espace de travail.
- Déploiement facilité: Simplifie la mise en ligne du projet sur le serveur

Le but principal est de pouvoir reproduire les environnements : faire un Dockerfile personnalisé, pour que n'importe qui le réutilise pour build son environnement grâce à cet immuable snapshot. On met cette image en ligne pour que n'importe qui puisse la récupérer et générer son propre container.

Ensuite, si le projet doit être une infrastructure composée de plusieurs applications, on pourra toutes les conteneuriser et les orchestrer pour qu'elles cohabitent dans un même environnement, malgré leurs différences.

C'est faisable en utilisant Docker Compose, mais on a également des outils tels que Kubernetes pour faire ce travail. Kubernetes simplifie la gestion des applications, améliore la disponibilité et facilite la mise à l'échelle, le tout grâce à une automatisation intelligente.

Imaginez **Kubernetes** comme un chef d'orchestre pour vos applications. Vous avez plusieurs applications emballées dans des conteneurs, et Kubernetes s'occupe de les déployer, de les mettre à l'échelle (ajuster automatiquement le nombre de copies en fonction de la demande), et de s'assurer qu'elles fonctionnent correctement. Il gère également la répartition du trafic entre les différentes parties de votre application, garantissant une disponibilité continue.

Docker Swarm est une autre technologie liée à la gestion de conteneurs, mais elle se concentre spécifiquement sur l'orchestration de conteneurs Docker. Contrairement à Kubernetes, qui est plus complexe et offre une gamme plus étendue de fonctionnalités, Docker Swarm est une solution d'orchestration plus légère et plus simple, développée directement par Docker.

Nous verrons cela plus tard.

Pour l'instant, comprenons-mieux comment s'articule un environnement Docker seul.

L'architecture Docker : qui parle à qui

Quand on tape une commande `docker`, elle ne s'exécute jamais toute seule : elle traverse toute une chaîne d'outils, chacun avec un rôle précis. C'est exactement la chaîne qu'on a reconstituée à la main dans la section précédente, avec des noms officiels cette fois.

Instant Notions : les 4 maillons de la chaîne Docker

- **Docker CLI (Command Line Interface)** : le programme `docker` qu'on tape dans le terminal. Il ne fait quasiment rien lui-même : il traduit la commande en requête HTTP et l'envoie plus loin.
- **`dockerd`** : le démon Docker, un processus qui tourne en permanence en arrière-plan, reçoit les requêtes du CLI (via un socket Unix ou une API réseau), et gère les images, les volumes, les networks.
- **`containerd`** : le runtime de haut niveau auquel `dockerd` délègue la gestion du cycle de vie des conteneurs (démarrage, arrêt, supervision), et qui gère aussi le téléchargement des images depuis un registry.
- **`runc`** : le runtime bas niveau qu'on a déjà croisé en creusant plus haut, celui qui appelle réellement le noyau pour créer les namespaces, poser les cgroups, et démarrer le processus du conteneur.

À l'autre bout de la chaîne, il y a le **registry** : un serveur qui stocke et distribue des images Docker. Le plus connu est [Docker Hub](#), le registry public par défaut, mais une entreprise peut très bien héberger son propre registry privé.

Donc quand on tape `docker run nginx` : le CLI transmet la demande à `dockerd`, `dockerd` vérifie si l'image `nginx` existe déjà localement, sinon demande à `containerd` de la télécharger depuis Docker Hub, puis `containerd` déclenche `runc` pour effectivement démarrer le conteneur avec les namespaces et cgroups qu'on a manipulés à la main juste avant.

Docker : Les trois termes les plus importants !

Dockerfile :

Script qui décrit les étapes de construction d'une image Docker.

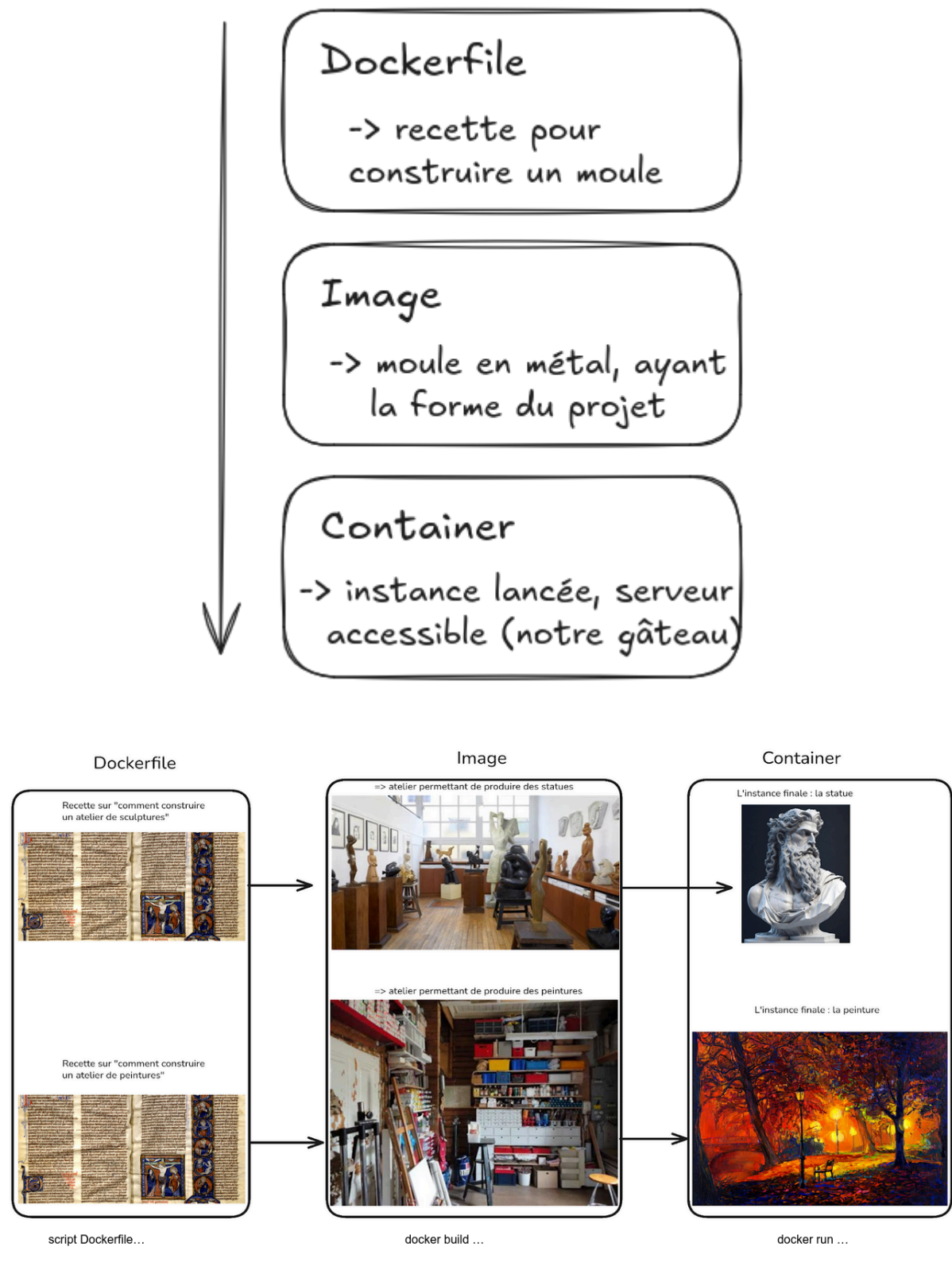
Image :

Snapshot (ou instantané de stockage, permet de réaliser une copie de données stockées sur un système de stockage, ou une copie des modifications apportées à ces données) immuable et exécutable d'une application et de son environnement. (**template for running containers**)

Container :

Instance en cours d'exécution d'une image Docker, isolée et portable. (running process)

Visuellement : Dockerfile, Image, Container



Mais j'ai pas compris, c'est quoi une image ?

C'est comme un modèle ou un "plan" qui contient tout ce qu'il faut pour faire fonctionner une application : le code, les bibliothèques, les dépendances et les configurations nécessaires

Une image est en lecture seule et ne change pas : elle sert simplement de base.

À partir de cette image, Docker peut créer un conteneur (une instance en cours d'exécution de l'image), c'est-à-dire un environnement isolé où l'application tourne vraiment

Cela permet de garantir que l'application marchera toujours pareil, peu importe où on la lance. On crée souvent une image à partir d'un fichier spécial appelé Dockerfile, qui décrit étape par étape comment construire l'image.

Les six notions à connaître par cœur

Le trio Dockerfile / Image / Container est le cœur du réacteur, mais trois autres notions complètent le programme du jour. On les pose maintenant en une ligne chacune, deux d'entre elles reviendront en profondeur un peu plus tard dans la journée.

Instant Notions : les 6 concepts de base de Docker

- **Container** : une instance en cours d'exécution d'une image, isolée par namespaces et cgroups, comme on vient de le voir.
- **Image** : le template immuable en lecture seule à partir duquel on crée des containers.
- **Registry** : le serveur qui stocke et distribue des images (Docker Hub en est l'exemple public par défaut).
- **Dockerfile** : le script texte qui décrit, étape par étape, comment construire une image.
- **Volume** : un mécanisme de stockage géré par Docker, en dehors du système de fichiers en couches du conteneur, qui permet aux données de survivre à la suppression du conteneur. On y reviendra en détail plus loin dans le programme.
- **Network** : le mécanisme qui permet à des conteneurs de communiquer entre eux et avec l'extérieur, en s'appuyant sur les namespaces réseau vus juste avant. On l'approfondit aussi plus tard dans la journée.

On prend les commandes en main

Assez de théorie, place au terminal. L'objectif : tourner autour du cycle de vie d'un conteneur avec `hello-world`, puis avec `nginx`, en comprenant bien ce que fait chaque commande, et surtout ce qui les distingue les unes des autres.

Vérification de la présence de Docker

```
# Vérification que Docker et Docker Compose sont bien installés
docker --version
docker compose version

# Test rapide : télécharge et lance une image minimaliste qui affiche un
message de bienvenue
docker run hello-world
```

Côté Docker Desktop

L'application est relativement explicite : sur notre gauche on constate les onglets pour voir les Conteneurs, les Images, les Volumes (qu'on approfondit plus tard aujourd'hui), etc.

`docker pull` : télécharger sans lancer

Qu'est-ce qui se passe si on veut juste récupérer une image, sans démarrer de conteneur tout de suite ? C'est exactement ce que fait `docker pull` : il va chercher l'image sur le registry (Docker Hub par défaut) et la stocke localement, sans rien exécuter.

```
docker pull nginx
```

Devrait afficher : une série de lignes `Pulling fs layer` / `Download complete` / `Pull complete` pour chaque couche de l'image, puis un résumé `Status: Downloaded newer image for nginx:latest`.

Pour obtenir la liste des images qu'on a en local, faire la commande `docker images`. On voit une liste d'images, dont celle qu'on vient tout juste de télécharger :

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	9b45b64f2f6e	3 weeks ago	192MB
hello-world	latest	1b44b5a3e06a	14 months ago	10.1kB

Maintenant il suffit de copier le code présent en colonne "IMAGE ID" si on veut cibler précisément cette image dans une commande.

Instant ménage

Parmi les lignes listées, on voit un détail qui pourra vous servir dans le futur : la colonne Size. Il se peut qu'un jour la mémoire de votre ordi soit trop rempli / pollué. Le détail se trouve plus loin dans le cours.

`docker create` puis `docker start`, versus `docker run`

Deux façons d'obtenir un conteneur en cours d'exécution existent, et il faut sentir la différence. `docker create` fabrique un conteneur à partir d'une image, mais ne le démarre pas : c'est comme préparer une voiture dans le garage, moteur éteint. `docker start` démarre ensuite un conteneur déjà créé (ou déjà arrêté).

```
# On crée le conteneur, sans le démarrer : on lui donne un nom pour le retrouver facilement
docker create --name mon-nginx -p 5000:80 nginx

# On le démarre séparément
docker start mon-nginx
```

Devrait afficher : pour `docker create`, un long identifiant hexadécimal (l'ID du conteneur créé). Pour `docker start`, le nom du conteneur, `mon-nginx`, en confirmation.

`docker run`, qu'on utilisait plus tôt avec `hello-world`, fait les deux étapes d'un coup : il crée le conteneur ET le démarre immédiatement. C'est le raccourci qu'on utilise 95% du temps, mais savoir qu'il cache un `create` + `start` aide à comprendre des messages d'erreur du type "conteneur déjà créé" quand on relance deux fois la même commande avec le même `--name`.

Observer ce qui tourne : `docker ps`

Pour obtenir les instances docker lancées, exécuter la commande `docker ps` :

```
docker ps
```

Devrait afficher : une ligne pour `mon-nginx`, avec sa colonne `STATUS` marquée `Up X seconds`. Le conteneur `hello-world` lancé tout à l'heure, lui, n'apparaît **pas** : il a déjà terminé son travail (afficher un message) et s'est arrêté. Pour voir aussi les conteneurs arrêtés, il faut ajouter `-a` :

```
docker ps -a
```

Devrait afficher : cette fois `hello-world` apparaît aussi, avec un `STATUS` du type `Exited (0) 5 minutes ago`. C'est un piège classique en début d'apprentissage : "j'ai lancé un conteneur mais `docker ps` ne le montre pas" veut souvent juste dire qu'il s'est déjà arrêté, pas qu'il a disparu.

`docker logs` et `docker exec`

Pour voir ce qu'un conteneur a affiché depuis son démarrage, sans y entrer :

```
docker logs mon-nginx
```

Devrait afficher : les logs d'accès et d'erreur de nginx, généralement vides juste après le démarrage tant qu'aucune requête n'est arrivée.

Pour exécuter une commande **à l'intérieur** d'un conteneur déjà en cours d'exécution, sans passer par `nsenter` à la main cette fois :

```
docker exec -it mon-nginx sh
```

Devrait afficher : un prompt `#` à l'intérieur du conteneur nginx. On peut taper `ls` `/usr/share/nginx/html` pour voir les fichiers servis par défaut, puis `exit` pour ressortir sans arrêter le conteneur.

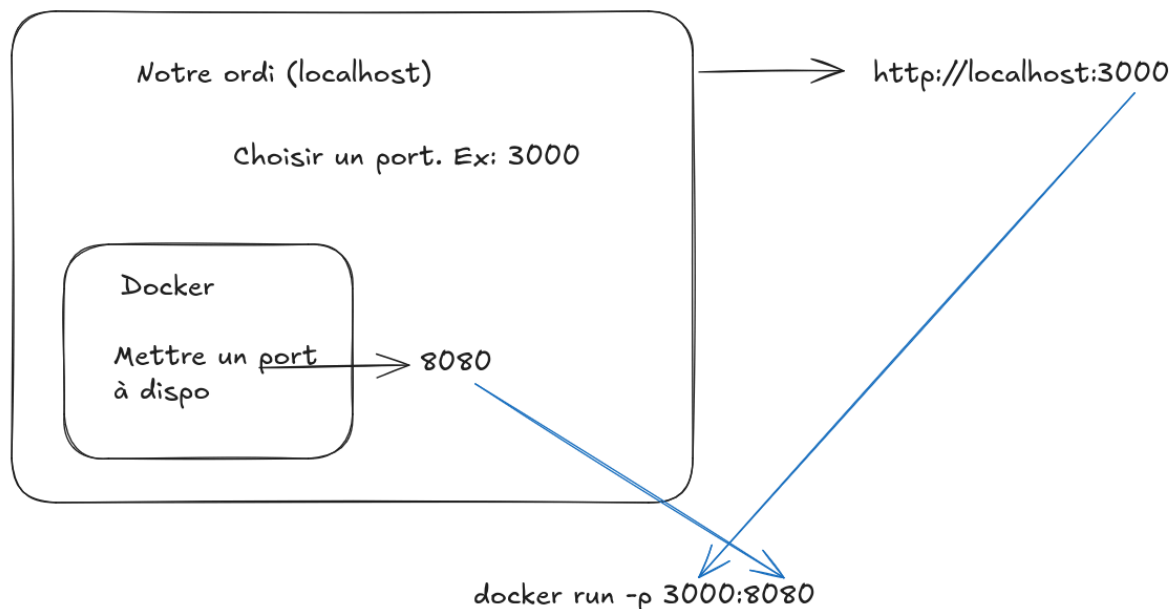
Et si l'on veut tout combiner en une seule commande, pour create + start + lancer le terminal dans notre container :

```
docker run -it --name mon-nginx -p 5000:80 nginx sh
```

Exposer les bons ports

Attends, je dois exposer 2 ports différents ?

Docker nous permet de lancer un environnement virtuel, isolé au sein de notre ordi. Pour qu'il soit accessible au port local de notre ordi, on va exposer ce fameux environnement virtuel sur un port pour docker



Donc la bonne méthode, c'est de lancer :

```
docker run -p 5000:80 nginx
```

- 5000 : c'est le port sur notre machine
- 80 : c'est le port à l'intérieur du conteneur (nginx écoute sur le port 80 par défaut)
- le nom juste après : celui de l'image, trouvée grâce à la commande `docker images` lancée dans le terminal
- maintenant ouvrir localhost:5000 sur navigateur, et cela devrait fonctionner !

Comment désactiver les ports ?

Vous l'aurez peut-être constaté, si on ferme la page, le projet continue de tourner.

Il faut donc :

- soit aller dans Docker Desktop pour désactiver les ports
- soit stopper le processus depuis le terminal

```
# Lister l'activité Docker
docker ps

# Stopper
docker stop NOM_DU_CONTENEUR
```

`docker stop` versus `docker rm`

Une fois qu'on a stoppé le conteneur avec `docker stop`, est-ce qu'il a disparu ? Non : `docker stop` arrête proprement le processus principal du conteneur (il envoie un signal `SIGTERM`, laisse quelques secondes, puis `SIGKILL` si besoin), mais le conteneur existe toujours, à l'état "exited", avec tout son système de fichiers en couche d'écriture intact. On peut d'ailleurs le relancer avec `docker start` sans rien perdre.

SIGTERM et SIGKILL ? Deux signaux que Linux envoie à un processus pour lui demander de s'arrêter. `SIGTERM` est poli : il prévient le programme, qui peut fermer ses connexions et finir ce qu'il fait avant de rendre la main. `SIGKILL` est brutal : le noyau tue le processus sur place, sans lui laisser la moindre chance de se préparer. D'où l'ordre choisi par Docker : on demande gentiment, puis on force.

Pour supprimer réellement le conteneur (son état, son système de fichiers en couche d'écriture, ses métadonnées), il faut `docker rm` :

```
docker stop mon-nginx
docker rm mon-nginx
```

Devrait afficher : le nom du conteneur en confirmation à chaque commande. Après le `rm`, `docker ps -a` ne montre plus `mon-nginx` du tout.

`docker rm` versus `docker rmi`

Dernier piège classique de vocabulaire : `docker rm` supprime un **conteneur**. `docker rmi` (remove image) supprime une **image**. Confondre les deux donne des messages d'erreur qui semblent absurdes au début.

```
# Supprime l'image nginx en local (le template), pas un conteneur en
particulier
docker rmi nginx
```

Devrait afficher : une erreur du type `image is being used by stopped container` si un conteneur (même arrêté) référence encore cette image. C'est volontaire : Docker refuse de supprimer une image tant qu'un conteneur, même éteint, en dépend encore, pour éviter de casser un `docker start` ultérieur. Il faut d'abord supprimer le conteneur avec `docker rm`, puis l'image avec `docker rmi`.

Que se passe-t-il si on tente `docker rmi` sur une image utilisée par un conteneur qui tourne encore, cette fois activement ? Le refus est encore plus strict : impossible de supprimer l'image tant que le conteneur n'est ni stoppé, ni supprimé.

Instant Cheat Sheet !

Le cycle de vie complet d'un conteneur

Un conteneur Docker traverse un nombre d'états limité, et connaître leurs noms exacts aide à lire `docker ps -a` sans deviner :

Instant Notions : les états d'un conteneur

- **created** : le conteneur existe (après `docker create`), mais n'a jamais tourné.
- **running** : le processus principal du conteneur est actif.
- **paused** : le conteneur est gelé (`docker pause`), tous ses processus sont suspendus par le noyau sans être arrêtés, comme une VM mise en veille.
- **exited** : le processus principal s'est arrêté, proprement (`docker stop`) ou de lui-même (fin de programme, crash).

- **dead** : un état plus rare, atteint quand Docker n'a pas réussi à arrêter proprement le conteneur, généralement après une erreur du daemon lui-même.

docker inspect : la fiche d'identité complète

Pour aller au-delà du résumé de `docker ps`, `docker inspect` sort toute la configuration d'un conteneur ou d'une image au format JSON : adresse IP interne, variables d'environnement, montages de volumes, politique de redémarrage, tout y est.

```
docker inspect mon-nginx
```

Devrait afficher : un objet JSON de plusieurs centaines de lignes. Pour ne récupérer qu'un champ précis sans tout lire à la main, on peut filtrer avec `--format` :

```
docker inspect --format='{{.State.Status}}' mon-nginx
```

Devrait afficher : juste `running` (ou l'état courant), sans le reste du JSON.

Faire le ménage : docker system df et docker system prune

Après quelques jours de manipulation, le disque se remplit d'images, de conteneurs arrêtés et de couches inutilisées. `docker system df` donne un état des lieux :

```
docker system df
```

Devrait afficher un tableau avec, par catégorie (Images, Containers, Local Volumes, Build Cache), l'espace total utilisé et l'espace réellement récupérable.

Exemple :

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	52	14	18.66GB	8.17GB (43%)
Containers	14	11	5.309MB	9.436kB (0%)
Local Volumes	17	3	683.8MB	317MB (46%)
Build Cache	61	0	71.56MB	13.06MB

Pour nettoyer, `docker system prune` supprime tout ce qui n'est plus utilisé par aucun conteneur actif : conteneurs arrêtés, réseaux orphelins, images sans conteneur associé, cache de build.

```
docker system prune
```

Devrait afficher une demande de confirmation (`y/N`), puis la liste de ce qui a été supprimé et l'espace récupéré. 🙌 Ajouter `-a` supprime en plus **toutes** les images non utilisées par un conteneur actif, pas seulement celles sans aucun conteneur du tout : à manier avec prudence sur une machine partagée.

Pour contrôler ça de manière plus granulaire :

Une bonne partie des Gb perdus sera due à des images, containers, volumes... non utilisés qui polluent votre disque dur. Voilà comment les nettoyer :

```
docker builder prune -af      # reclaims build cache (always safe)
docker image prune -f         # removes dangling (untagged) images only
docker container prune -f     # removes STOPPED containers only
```

D'autres commandes, un peu plus dangereuses, à vérifier d'abord (ne pas supprimer quelque chose par accident) :

```
docker system prune -a        # Δ removes ALL images not used by a
running container             # → any stopped project's images will be
wiped
docker volume prune -a        # Δ removes unused volumes → DATA LOSS if
a DB volume                   # isn't currently attached to a running
container
```

Ce qui casse quand on oublie `-d`

Sans l'option `-d` (detached, en arrière-plan), `docker run` attache le terminal au conteneur : le prompt reste bloqué tant que le conteneur tourne, et fermer le terminal (ou faire `Ctrl+C`) envoie un signal d'arrêt au conteneur. Beaucoup de "mon conteneur s'est arrêté tout seul" viennent de là. Pour laisser tourner nginx en tâche de fond, on tape plutôt :

```
docker run -d -p 5000:80 --name mon-nginx nginx
```

Les pièges de nommage et du tag `latest`

Deux derniers réflexes à prendre tout de suite. D'abord, donner un nom explicite à ses conteneurs avec `--name` plutôt que de laisser Docker générer un nom aléatoire du type `hopeful_turing` : ça évite de chercher un ID à rallonge dans `docker ps` à chaque commande.

Ensuite, se méfier du tag `latest`. Si on ne précise pas de tag après le nom d'une image (`nginx` plutôt que `nginx:1.27`), Docker utilise `latest` par défaut. Le problème : `latest` n'a **aucune signification particulière**, c'est juste un tag comme un autre que les mainteneurs d'une image mettent généralement à jour vers leur dernière version stable, mais rien ne le garantit techniquement. Deux `docker pull nginx` à deux mois d'écart peuvent très bien récupérer deux versions différentes sans qu'on s'en rende compte, ce qui casse la reproductibilité qu'on cherche justement à obtenir avec Docker. Le bon réflexe en production : toujours épingler une version précise, `nginx:1.27-alpine` plutôt que `nginx`.

On construit notre première image

Jusqu'ici, tous les conteneurs qu'on a lancés venaient d'images écrites par quelqu'un d'autre : `hello-world`, `nginx`. Il est temps de fabriquer la nôtre. Et cette toute petite application va servir de point de départ au projet de l'après-midi : autant la construire ensemble, une fois, pour que personne ne parte avec un trou dans les fondations.

On crée le projet Node.js

On part d'un dossier vide, avec son propre dépôt git et son `package.json` :

```
mkdir projet-docker
cd projet-docker

git init
npm init
```

`npm init` pose une série de questions (nom, version, description...) : les valeurs par défaut conviennent très bien pour aujourd'hui, il suffit de valider avec `Entrée` à chaque ligne. On installe ensuite [Express](#), le framework qui simplifie l'écriture de routes HTTP en Node.js :

```
npm install express
```

Le `package.json` généré doit contenir un script `start`, sinon rien de ce qui suit ne fonctionnera : c'est cette ligne que `npm start` ira chercher tout à l'heure, et c'est aussi elle que le conteneur exécutera plus tard.

```
{
  "name": "mon-app",
  "version": "1.0.0",
  "description": "Ma toute premiere app Node conteneurisee",
  "main": "src/index.js",
  "scripts": {
    "start": "node src/index.js"
  },
  "dependencies": {
    "express": "^4.19.2"
  }
}
```

`node_modules` ne doit jamais être versionné avec git : trop volumineux, et de toute façon reconstituable à tout moment avec `npm install`. On pose donc un `.gitignore` avant d'aller plus loin :

```
node_modules
```

Reste à créer le fichier qui contient réellement l'application :

```
mkdir src
touch src/index.js
```

Et on y écrit ce code, une seule route qui répond en JSON :

```
const app = require("express")();

app.get("/", (req, res) => res.json({ message: "Bonjouuuur :)" }));

const port = process.env.PORT || 3000;

app.listen(port, () =>
  console.log(`app listening on http://localhost:${port}`)
);
```

On vérifie que l'app tourne, sans Docker

Avant même de penser à Docker, on s'assure que l'app tourne toute seule, sur la machine, avec la commande que le `package.json` vient de définir :

```
npm start
```

Devrait afficher :

```
> mon-app@1.0.0 start
> node src/index.js

app listening on http://localhost:3000
```

En ouvrant `http://localhost:3000` dans le navigateur, on doit voir la réponse JSON `{"message": "Bonjouuuur :)"}`. Une fois la vérification faite, on arrête le serveur (`Ctrl+C`) : l'objectif maintenant, c'est de faire tourner exactement la même chose, mais depuis un conteneur.

Le Dockerfile minimal

À la racine du projet, dans un fichier nommé `Dockerfile` (sans extension), on écrit ceci :

```
# Image officielle qui embarque déjà Node.js et npm :
https://hub.docker.com/_/node
FROM node:18

# Tout ce qui suit (COPY, RUN, CMD) s'exécute depuis ce dossier, à l'intérieur
du conteneur
WORKDIR /app

# On copie d'abord les fichiers qui décrivent les dépendances : RUN npm install
en a besoin
COPY package*.json ./

# Installation des dépendances, dans l'environnement Linux du conteneur, pas
celui de la machine
RUN npm install

# Le reste du code source arrive maintenant
COPY . .
```

```
# On documente le port sur lequel l'app écoute à l'intérieur du conteneur
EXPOSE 3000

# Démarre le conteneur avec la même commande qu'on vient de tester à la main
CMD ["npm", "start"]
```

Le `.dockerignore`

Un deuxième fichier à poser à côté du premier, `.dockerignore` : le même principe que le `.gitignore` de tout à l'heure, mais cette fois pour ce que `docker build` envoie à Docker (le contexte de build). Sans lui, `node_modules` partirait aussi dans l'image, alors que la ligne `RUN npm install` du Dockerfile va de toute façon le reconstruire de zéro à l'intérieur du conteneur.

```
node_modules
```

On construit l'image et on lance le conteneur

Place à la construction de l'image, avec `docker build` et un nom explicite pour la retrouver facilement :

```
docker build -t mon-app .
```

Devrait afficher : une série d'étapes numérotées, une par instruction du Dockerfile (`FROM`, `WORKDIR`, `COPY` ...), puis une ligne finale du type `Successfully tagged mon-app:latest`.

Reste à lancer un conteneur à partir de cette image fraîchement construite, avec le mapping de port qu'on connaît déjà :

```
docker run -p 3000:3000 mon-app
```

Devrait afficher : sans `-d`, le terminal reste attaché et affiche en direct `app listening on http://localhost:3000`, exactement comme tout à l'heure en local, sauf que cette fois la commande tourne à l'intérieur du conteneur. Depuis un autre terminal, on peut vérifier :

```
curl http://localhost:3000
```

Devrait afficher :

```
{ "message": "Bonjouuuur :)" }
```

La même réponse est aussi visible en ouvrant `http://localhost:3000` dans le navigateur. 😎

Le piège numéro un : des ports qui ne se parlent pas

Le conteneur tourne, `docker ps` l'affiche même en `Up`, mais le navigateur n'affiche jamais rien et `curl` reste bloqué ou renvoie une erreur de connexion. D'où ça vient ?

Trois nombres doivent désigner le même port, et un seul suffit à tout casser s'il diverge :

- le port sur lequel l'app écoute réellement dans `src/index.js` (`process.env.PORT || 3000`)

- le port documenté par `EXPOSE` dans le Dockerfile
- le port de droite dans `-p 3000:3000`, celui à l'intérieur du conteneur

Beaucoup de squelettes Node.js trouvés en ligne écoutent par défaut sur `8080` plutôt que `3000`. Si on copie ce genre de code sans y toucher, mais qu'on garde `EXPOSE 3000` et `-p 3000:3000`, le mapping pointe vers un port sur lequel, à l'intérieur du conteneur, rien n'écoute vraiment : le tout tourne, sans jamais planter, et pourtant rien ne répond jamais côté navigateur. `EXPOSE` ne fait d'ailleurs que documenter cette intention, il n'ouvre rien tout seul : le vrai mapping se joue entièrement au `-p`. La seule solution, c'est de vérifier que ces trois nombres racontent la même histoire.

Voilà, une image construite à la main, un conteneur qui tourne, une réponse JSON dans le navigateur : ce socle est prêt à être malmené, agrandi et corrigé cet après-midi.

Mise en pratique - Après-midi : le projet fil rouge Todo API

Le matin, on a suivi : DevOps, containers Linux, installation, architecture Docker, et une petite app Node.js "hello" qu'on a fait tourner ensemble, en live, dans un premier Dockerfile. Cet après-midi, le rythme change complètement : c'est fini de suivre, il est temps de construire. Chaque chapitre qui vient apporte une notion nouvelle, network, volume, Compose, registry... et le travail, c'est de la transposer soi-même sur son propre projet, pas de recopier un exemple. Let's go, on fait grandir cette app "hello" jusqu'à en faire une vraie stack !

Le projet

On part de l'app Node.js "hello" du matin, dockerisée à la truelle avec un Dockerfile minimal. En fin de journée, cette app est devenue une API de gestion de tâches, avec :

- une API de tâches (CRUD complet) branchée sur une base [PostgreSQL](#) dont les données survivent à un redémarrage,

PostgreSQL ? Un système de gestion de base de données relationnelle open source, l'un des plus utilisés en production.

- toute la configuration de l'application pilotée par des variables d'environnement, pas une ligne en dur dans le code,
- un réseau isolé, où la base de données n'est plus joignable depuis la machine hôte,
- un second service, écrit en [Python](#), le langage généraliste très utilisé côté data et outillage, branché sur la même base,
- les images de l'application publiées sur un [registry](#) puis redéployées sans une seule ligne de code source en local.

Registry ? Un serveur qui stocke des images Docker et les rend téléchargeables, l'équivalent d'un GitHub mais pour des images plutôt que du code.

En sortie de journée, on doit pouvoir remplir un tableau de métriques observables : la taille de chaque image, le temps de build à froid et à chaud, le temps jusqu'à la première réponse HTTP, et le nombre de couches de chaque Dockerfile.

Ce que vous allez rendre

Le projet démarré cet après-midi, c'est un repository Git **public**, qui contient à terme :

- un README complet, qui explique comment lancer le projet et ce qu'il fait,
- le code source de l'API Node.js : un CRUD qui manipule des `Task`, chacune avec au minimum un `id`, un `status` et une `description`,
- une dockerisation complète du projet, avec les bonnes pratiques vues chapitre après chapitre.

Nouveauté par rapport à ce qu'on connaît déjà : le README contient une section **Journal de bord**, avec une entrée par chapitre. Chaque entrée note ce qui a été mesuré et ce qui a cassé, et pourquoi. C'est là, par écrit, que se répondent les questions posées dans chaque chapitre. Cette section est notée au même titre que le code.

Voilà à quoi ressemble une entrée bien remplie, pour fixer le format :

Le socle : CRUD testé avec les 3 cas demandés, tous passent. Premier accroc : un `POST` avec une `description` de 50 000 caractères faisait planter le process, avant l'ajout d'une limite de taille explicite dans la validation.

Je le rappelle : le dépôt est **noté sur ses commits autant que sur son résultat**. Des commits fréquents et atomiques, un commit égale un changement logique, jamais de `git add .` en aveugle mais un ajout fichier par fichier, une branche dès que plusieurs sujets avancent en parallèle.

Le socle : la Todo API

Avant de coder : le dépôt se note sur ses commits autant que sur son résultat final. Un commit dès que les routes répondent, un autre quand la validation des entrées est en place : deux changements logiques, deux commits, chacun avec ses fichiers ajoutés explicitement.

Il est temps de transformer l'app "hello" du matin en un vrai CRUD de tâches. On commence volontairement sans aucune base de données : un tableau en mémoire suffit pour valider les routes avant de brancher quoi que ce soit de persistant. On construit avec [Express](#), le framework HTTP minimaliste sur lequel tourne l'immense majorité des API Node.js en production.

Express ? Une couche fine au-dessus de Node.js qui gère le routage HTTP (quelle fonction répond à quelle URL) et les middlewares, des fonctions qui interceptent chaque requête, par exemple pour la validation ou la gestion d'erreurs.

Structure du projet visée

```
todo-api/  
├─ src/  
│   ├─ routes/  
│   │   └─ tasks.js  
│   ├─ models/  
│   │   └─ task.js  
│   ├─ middleware/  
│   │   └─ errorHandler.js  
│   └─ app.js  
├─ tests/                                     # on s'en servira plus tard dans la semaine  
└─ unit/
```



```

|   |   └─ task.test.js
|   └─ integration/
|       └─ api.test.js
└─ Dockerfile
└─ .dockerignore
└─ .gitignore
└─ package.json
└─ README.md

```

Les 5 routes REST, sous `/api/tasks`

CRUD et REST ? CRUD, c'est l'acronyme des quatre opérations de base sur une donnée : Create, Read, Update, Delete. REST (Representational State Transfer) est le style d'architecture qui les expose en HTTP, où chaque URL désigne une ressource et où c'est le verbe HTTP (`POST`, `GET`, `PUT`, `DELETE`) qui dit ce qu'on veut en faire.

- `POST /api/tasks` : créer une tâche
- `GET /api/tasks` : lister toutes les tâches
- `GET /api/tasks/:id` : voir une tâche
- `PUT /api/tasks/:id` : modifier une tâche
- `DELETE /api/tasks/:id` : supprimer une tâche

Le modèle de données d'une `Task`

UUID ? Universally Unique Identifier. Un identifiant généré aléatoirement, assez long pour qu'on puisse en créer des milliards sans jamais tomber deux fois sur le même. Pratique ici : chaque tâche reçoit son `id` sans avoir besoin d'un compteur central qui saurait où on en est.

```

{
  id: "uuid",
  description: "string",
  status: "string",
  createdAt: "timestamp",
  updatedAt: "timestamp"
}

```

`src/app.js`, le câblage de l'application

Voilà le fichier qui assemble tout : les middlewares de sécurité et de parsing JSON, une route `/health` pour vérifier que le serveur répond, le montage des routes de tâches, et un gestionnaire d'erreurs central.

Deux de ces middlewares méritent un mot, puisqu'ils arrivent sans prévenir dans les imports. [helmet](#) positionne une série d'en-têtes HTTP de sécurité que les navigateurs savent lire, et [cors](#) autorise (ou refuse) les appels venant d'un autre domaine que celui qui sert l'API.

CORS ? Cross-Origin Resource Sharing. La règle du navigateur qui empêche, par défaut, une page servie par un domaine d'appeler une API hébergée sur un autre domaine. Le middleware sert à ouvrir explicitement cette porte quand c'est voulu.

```
const express = require('express');
```

```

const cors = require('cors');
const helmet = require('helmet');
const taskRoutes = require('./routes/tasks');
const errorHandler = require('./middleware/errorHandler');

const app = express();

// Middleware
app.use(helmet());
app.use(cors());
app.use(express.json());

// Health check
app.get('/health', (req, res) => {
  res.json({ status: 'ok', timestamp: new Date() });
});

// Routes
app.use('/api/tasks', taskRoutes);

// Error handling
app.use(errorHandler);

module.exports = app;

```

Reste à écrire trois fichiers, et c'est le travail de cette phase :

- `src/routes/tasks.js` : les 5 routes ci-dessus, qui appellent le modèle et renvoient du JSON.
- `src/models/task.js` : le stockage en mémoire des tâches, un simple tableau suffit pour l'instant.
- `src/middleware/errorHandler.js` : un middleware qui transforme toute erreur en réponse JSON propre, jamais un stacktrace brut envoyé au client.

Trois cas à valider avant d'enchaîner sur la suite

- Une création suivie d'un `GET /api/tasks` renvoie bien la tâche créée, avec un `id` généré.
- Un `GET /api/tasks/:id` sur un identifiant inexistant renvoie un `404` propre, jamais un crash du process.
- Un `POST` avec un corps JSON malformé, ou une `description` de plusieurs dizaines de milliers de caractères, est refusé avec un `400` clair, sans jamais faire tomber le serveur.

La grille de notation

- 10% : API CRUD fonctionnelle et robuste aux entrées invalides
- 15% : Dockerfile de production (image épinglée, multi-stage, utilisateur non-root, `.dockerignore`)
- 10% : persistance PostgreSQL via volume nommé, données qui survivent à un redémarrage
- 10% : isolation réseau (network custom, base non exposée sur l'hôte)
- 10% : configuration entièrement externalisée (`.env` non commité, `.env.example` présent, rien en dur)

- 10% : stack complète pilotée par docker-compose, démarrable en une commande
- 10% : service Python de statistiques branché sur la stack et robuste
- 15% : images publiées sur un registry et stack redéployable sans code source local
- 5% : tableau de métriques rempli, optimisations mesurées et justifiées
- 5% : rigueur du dépôt (commits atomiques et fréquents, README et journal de bord à jour)

5 - Écrire un Dockerfile de production

Ce matin, on a écrit un Dockerfile en quelques minutes : une image de base, un `WORKDIR`, une copie des fichiers, un `npm install`, un `CMD`. Il a suffi à faire tourner l'app hello, puis la Todo API qu'on vient de transformer en CRUD complet. C'est un bon brouillon, pas un Dockerfile de production. À partir d'ici, l'après-midi passe en autonomie totale : chacun avance sur son propre projet, à son rythme. On va comprendre ce qui distingue un Dockerfile qui "marche sur ma machine" d'un Dockerfile qu'on peut réellement déployer, et pourquoi l'ordre des lignes à l'intérieur n'a rien d'anodin. Et on le fait maintenant, avant de brancher plusieurs conteneurs entre eux et d'orchestrer toute la stack : une image mal construite propage ses défauts partout où elle est réutilisée, autant la solidifier à la source.

L'essentiel du Dockerfile de production

Le [Dockerfile](#) est un script texte qui contient une série d'instructions permettant de construire une image Docker. Il spécifie l'environnement de travail, les dépendances, les étapes nécessaires à la configuration d'une application à l'intérieur d'un conteneur Docker : sélection d'une image de base, définition du répertoire de travail, copie des fichiers du projet, installation des dépendances, etc.

Les instructions qui comptent

- `FROM` : l'image de base sur laquelle on construit. Toujours la première instruction (sauf `ARG` avant, cas particulier qu'on verra plus loin).
- `WORKDIR` : le répertoire de travail à l'intérieur du conteneur. Toutes les commandes suivantes (`COPY`, `RUN`, `CMD`) s'exécutent depuis là.
- `COPY` : copie des fichiers depuis notre machine (le build context) vers l'image.
- `ADD` : fait la même chose que `COPY`, mais avec des super-pouvoirs en plus (extraction automatique d'archives `.tar`, récupération d'une URL distante). Le souci, c'est que ces super-pouvoirs sont rarement ce qu'on veut, et ils rendent le comportement moins prévisible. La bonne pratique, c'est `COPY` par défaut, et `ADD` seulement quand on a explicitement besoin d'extraire une archive locale.
- `RUN` : exécute une commande pendant le build (installer des paquets, compiler, etc.). Chaque `RUN` crée une nouvelle couche.
- `ENV` : définit une variable d'environnement, disponible au build et au runtime.
- `ARG` : définit une variable disponible seulement pendant le build (pas dans le conteneur final). Pratique pour paramétrer un build sans polluer le runtime.

- `EXPOSE` : documente le port sur lequel l'application écoute. Attention, ça ne publie rien tout seul : c'est de la documentation pour qui lit le Dockerfile, le vrai mapping de port se fait au `docker run -p`.
- `USER` : change l'utilisateur qui exécute les instructions suivantes et le processus final. Par défaut, sans `USER`, tout tourne en `root` à l'intérieur du conteneur. On y revient juste après.
- `CMD` vs `ENTRYPOINT` : les deux définissent ce qui se lance au démarrage du conteneur. `CMD` donne une commande par défaut, qu'on peut écraser facilement au `docker run (docker run mon-image autre-commande)`. `ENTRYPOINT` fixe la commande de façon plus rigide : ce qu'on passe au `docker run` s'ajoute en argument plutôt que de remplacer. En pratique, la combinaison la plus courante est `ENTRYPOINT` pour fixer le binaire, `CMD` pour donner les arguments par défaut. Pour nos Dockerfiles du jour, un `CMD` seul suffit.
- `HEALTHCHECK` : permet à Docker de vérifier périodiquement que le conteneur est vraiment en bonne santé (pas juste "démarré", mais "répond correctement"). On creusera ça plus tard avec le monitoring, mais on peut déjà en glisser un dans nos Dockerfiles.

Build context ? L'ensemble des fichiers du dossier (et sous-dossiers) envoyés au démon Docker au moment du `docker build`. C'est pour ça qu'on ne doit jamais lancer `docker build` depuis un dossier contenant des gigas de fichiers inutiles : tout part dans le context, même ce qu'on ne `COPY` pas explicitement.

Les layers et le cache de build

Chaque instruction d'un Dockerfile (`RUN`, `COPY`, `ADD`...) crée une couche (*layer*) dans l'image. Docker garde ces couches en cache : si une instruction et son contexte n'ont pas changé depuis le dernier build, Docker réutilise la couche telle quelle au lieu de la refaire. Et dès qu'**une** couche change, **toutes les couches suivantes** doivent être reconstruites, même si elles, individuellement, n'ont pas changé.

C'est là que l'ordre des instructions devient une vraie question de perf. Prenons un Dockerfile Node.js écrit "dans le désordre" :

Avant (mauvais ordre) :

```
FROM node:22-alpine
WORKDIR /app
COPY . .
RUN npm ci
CMD ["node", "src/server.js"]
```

Ici, `COPY . .` copie tout le projet, code source inclus. Résultat : à chaque fois qu'on modifie une seule ligne de code (même un `console.log` pour debug), la couche `COPY . .` change, donc le cache est invalidé, donc `RUN npm ci` doit se refaire en entier. Sur un projet avec des dizaines de dépendances, ça veut dire retélécharger et réinstaller tout `node_modules` à chaque modification de code. Sur une CI, ça peut transformer un rebuild de 5 secondes en rebuild de 2 minutes.

Après (bon ordre) :

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
CMD ["node", "src/server.js"]
```

Là, on copie d'abord uniquement `package.json` et `package-lock.json` (`package*.json` matche les deux). Tant qu'on ne touche pas aux dépendances, cette couche ne change pas, donc `RUN npm ci` reste en cache. Ce n'est qu'ensuite qu'on copie le reste du code avec `COPY . .`. Résultat concret : un rebuild après une modification de code source réutilise le cache jusqu'à `RUN npm ci` inclus, et ne refait que la copie du code (quasi instantané) plutôt que la réinstallation complète des dépendances.

💡 La règle générale : **on met le moins changeant en premier**. Les dépendances changent rarement (on ajoute un package une fois de temps en temps), le code source change à chaque commit. Donc dépendances avant, code après.

`.dockerignore` : ce qu'on oublie souvent

On l'a vu, `docker build` envoie tout le build context au démon Docker, et un `COPY . .` copie tout ce contexte dans l'image. Sans filtre, ça inclut potentiellement `node_modules/`, `.git/`, des fichiers `.env` avec des secrets, des logs...

Un fichier `.dockerignore` à la racine du projet fonctionne comme un `.gitignore` : chaque ligne exclut un fichier ou dossier du build context.

```
node_modules
.git
.env
*.log
npm-debug.log*
.DS_Store
dist
coverage
```

Qu'est-ce qui se passe concrètement si on oublie `node_modules` dans le `.dockerignore` ? Deux problèmes en cascade. D'abord, le build context grossit énormément (des dizaines voire centaines de Mo), donc chaque `docker build` commence par un envoi de contexte plus lent. Ensuite, et c'est pire : le `COPY . .` écrase le `node_modules` fraîchement installé par `RUN npm ci` avec celui de la machine hôte, potentiellement compilé pour un autre OS ou une autre architecture (typiquement : des modules natifs compilés sur un Mac qui ne fonctionnent pas dans le conteneur Linux). Résultat : l'appli plante au démarrage avec une erreur de binding natif, souvent difficile à diagnostiquer si on ne pense pas tout de suite au `.dockerignore`.

Les Images, plus en détail

Définition : l'image est un package léger et autonome qui contient tout le nécessaire pour exécuter une application, y compris :

- le code,
- les bibliothèques,

- les dépendances,
- les variables d'environnement,
- les fichiers de configuration

Les images sont construites à partir d'un ensemble d'instructions définies dans un Dockerfile. Elles sont immuables, ce qui signifie qu'une fois créées, elles ne sont pas modifiées. Les conteneurs Docker sont ensuite créés à partir de ces images et représentent l'instance en cours d'exécution de l'application.

Pour créer une image :

```
docker build -t NomDeMonImage source/de/fichiers

# Exemple:
docker build -t NomDeMonImage .

# Bonnes pratiques:
docker build -t mon_pseudo/nom-de-mon-projet:1.0 .
```

Erreur courante : l'oubli du `.` à la fin de la commande

- l'usage de `-t` permet de donner un nametag à l'image
- un bon réflexe de donner mon pseudo suivi du nom de l'app
- on remarque qu'il suit les étapes de mon Dockerfile

Devrait afficher : une ligne par instruction du Dockerfile, préfixée d'un numéro d'étape du type `[2/6] WORKDIR /app`, puis un `naming to docker.io/library/...` en fin de build. En relançant immédiatement la même commande, les étapes inchangées affichent `CACHED` : c'est le cache de couches qu'on vient de voir en action.

Ce qui fait qu'un Dockerfile est prêt pour la prod

Le Dockerfile de ce matin fait le strict minimum : il construit une image qui démarre. Un Dockerfile de prod ajoute une poignée d'instructions qui, une par une, ferment une faille ou évitent un piège classique. Le tableau résume chacune, avec ce qu'elle garantit et ce qui casse si on l'oublie.

Instruction	Ce qu'elle garantit	Ce qui casse sans elle
Image de base épinglée (<code>node:22.14.0-alpine</code> , jamais <code>node:latest</code>)	Le même Dockerfile produit la même image demain comme aujourd'hui	Un build qui marchait hier peut casser sans qu'on ait touché une ligne de code
<code>COPY package*.json</code> <code>./</code> avant <code>COPY . .</code>	Le cache de la couche d'installation survit à un changement du seul code applicatif	Chaque commit sur le code force une réinstallation complète des dépendances

Instruction	Ce qu'elle garantit	Ce qui casse sans elle
<code>npm ci --omit=dev</code> plutôt que <code>npm install</code>	Installation strictement reproductible depuis le lock file, sans les dépendances de dev en trop	Des versions qui dérivent d'une install à l'autre, une image gonflée d'outils inutiles en prod
<code>USER</code> (un utilisateur non privilégié)	Le process final tourne avec des droits limités	Sans <code>USER</code> , tout tourne en <code>root</code> : une faille applicative devient un accès root dans le conteneur
<code>EXPOSE <port></code>	Documente le port d'écoute pour quiconque lit le Dockerfile	Rien ne casse techniquement, mais l'information se perd, et un outil d'inspection d'image ne la retrouve nulle part
<code>HEALTHCHECK</code>	Docker (et l'orchestrateur, plus tard) distingue un conteneur "démarré" d'un conteneur qui répond vraiment	Un conteneur planté en boucle mais toujours affiché "Up", personne n'est alerté
<code>CMD</code> en forme exec (<code>["node", "app.js"]</code>)	Le process applicatif devient le PID 1 et reçoit correctement <code>SIGTERM</code> pour un arrêt propre	En forme shell, un shell intermédiaire prend le PID 1 et ne relaie pas toujours le signal : l'arrêt traîne ou coupe le process brutalement

Trois de ces instructions se lisent mieux avec un petit fragment sous les yeux :

```
# Utilisateur non privilégié : le process final n'a plus les droits root
USER node
```

```
# Forme exec (tableau JSON) : le process devient PID 1, il reçoit SIGTERM
correctement
CMD ["node", "app.js"]
```

```
# Sonde toutes les 30s que l'app répond vraiment, pas juste qu'elle a démarré
HEALTHCHECK --interval=30s --timeout=3s \
  CMD node -e "require('http').get('http://localhost:4000/health', r =>
process.exit(r.statusCode === 200 ? 0 : 1))"
```

Pourquoi `node:22-alpine` et pas `node:22` tout court ? [Alpine](#) est une distribution Linux ultra-légère (quelques Mo contre plusieurs centaines pour une Debian classique). Moins de composants installés, c'est aussi moins de surface d'attaque niveau sécurité.

Pourquoi épingler `node:22-alpine` plutôt que `node:latest` ou même `node:alpine` sans version ?

Parce que `latest` change dans le temps : le build qui marchait hier peut casser demain sans qu'on ait touché une ligne de code, simplement parce que l'image de base a changé sous nos pieds. Épingler une version précise (`22-alpine`, voire `22.14.0-alpine` pour être encore plus strict), c'est garantir que le même Dockerfile produit la même image, aujourd'hui comme dans six mois.

Le multi-stage build, le principe

Un Dockerfile en une seule étape garde dans l'image finale des choses dont on n'a plus besoin au runtime : les outils de build éventuels, le cache du gestionnaire de paquets, parfois des fichiers de config qui ne servent qu'à la compilation. Le [multi-stage build](#) résout ça : on utilise plusieurs `FROM` dans le même Dockerfile, chacun démarrant une étape (*stage*) indépendante, et on ne copie dans l'image finale que ce dont on a réellement besoin, via `COPY --from=<nom_du_stage>`.

Le mécanisme, réduit à l'essentiel :

```
FROM node:22.14.0-alpine AS builder
# ... installation complète et étape de build ...

FROM node:22.14.0-alpine
# On ne récupère que le résultat, rien des outils qui ont servi à le produire
COPY --from=builder /app/dist ./dist
```

Le premier stage installe tout, y compris les dépendances de dev, et fait le travail de build. Le second repart d'une image fraîche et ne rapatrie que le strict nécessaire : aucun outil de compilation, aucun cache de téléchargement, aucune trace du premier stage.

Le gain, c'est vraiment mesurable ou c'est juste une histoire de propreté ? Vérifions avec `docker images` après avoir buildé une version simple et une version multi-stage du même projet :

```
docker build -t mon-image:simple -f Dockerfile.simple .
docker build -t mon-image:multistage -f Dockerfile.multistage .
docker images | grep mon-image
```

Devrait afficher quelque chose comme :

REPOSITORY	TAG	IMAGE ID	SIZE
mon-image	simple	a1b2c3d4e5f6	187MB
mon-image	multistage	f6e5d4c3b2a1	142MB

Sur un projet Node.js simple sans étape de build, l'écart reste modeste (on élimine surtout le cache npm et les fichiers temporaires). Mais dès qu'il y a une vraie étape de compilation (TypeScript, Webpack, un framework front à builder), l'écart explose : le stage `builder` peut peser 500 Mo à 1 Go avec tous les outils de compilation, alors que le stage final ne garde que le résultat compilé et les dépendances de prod, souvent 5 à 10 fois plus petit.

Checklist "Dockerfile de prod"

Avant de considérer un Dockerfile comme prêt pour la production :

- image de base épinglée sur une version précise, jamais `latest`
- utilisateur non-root (`USER`)
- aucun secret copié dans une couche, même supprimé après (une couche supprimée reste dans l'historique de l'image, voir `docker history` plus loin)
- `.dockerignore` présent et à jour (`node_modules`, `.git`, `.env`, fichiers de logs...)
- multi-stage build dès qu'il y a une étape de compilation ou des outils de dev
- un seul processus par conteneur (pas de serveur web + base de données + cron dans le même Dockerfile : chaque service a le sien)
- `CMD` en forme exec (`[ "node", "src/server.js" ]`), jamais en forme shell (`"node src/server.js"`), pour une gestion correcte des signaux d'arrêt

À votre tour

Rappel notation : chaque phase franchie mérite son propre commit. Si le Dockerfile part en plusieurs itérations (un premier jet, puis une optimisation), ce sont deux commits distincts, pas un seul fourre-tout.

Le Dockerfile qu'on a écrit ce matin pour l'app hello, devenue Todo API, reste un brouillon de développement. Il est temps de le transformer en Dockerfile de production. Cinq choses doivent être vraies à la sortie, chacune vérifiable par une commande précise :

- L'image de base est épinglée sur une version précise. On vérifie en lisant la ligne `FROM` du Dockerfile : aucune trace de `latest` ni de tag flottant sans version.
- Le `.dockerignore` est complet. On liste la racine de l'image avec `docker run --rm mon-image ls -a`, et rien de ce qui n'a rien à faire là ne doit apparaître : ni `.git`, ni `.env`, ni fichier de log. Le second signe qui ne trompe pas, c'est la ligne `transferring context` en début de build : elle doit se compter en dizaines de kilo-octets, pas en dizaines de méga-octets.
- Le process final ne tourne pas en root. On vérifie avec `docker run --rm -it mon-image sh -c whoami`, qui ne doit jamais répondre `root`.
- Le build est multi-stage, et l'image finale ne contient plus les dépendances de développement. On le vérifie sur une dépendance qui n'a rien à faire en production, par exemple `docker run --rm mon-image ls node_modules | grep -c jest`, qui doit répondre `0` si un outil de test traîne dans le `package.json`.
- L'ordre des instructions protège le cache. On modifie uniquement un fichier de code applicatif, on relance le build, et la ligne d'installation des dépendances doit afficher `CACHED` dans la sortie.

Deux mesures à relever avant et après ce travail : la taille de l'image (`docker images`), et le temps de build à froid (`time docker build --no-cache -t mon-image .`) comparé au temps de build à chaud, une fois le cache chaud (`time docker build -t mon-image .`). Ces mesures rejoignent la section Journal de bord du README, aux côtés de celles des autres chapitres.

Le contrôle avant de passer à la suite

Trois scénarios à rejouer avant d'enchaîner :

- Nominal : l'image se build sans erreur, le conteneur démarre, et l'API répond correctement sur son endpoint habituel. C'est la preuve que le passage en Dockerfile de prod n'a rien cassé au passage.
- Limite, sur le cache de build : on modifie uniquement le code applicatif (jamais `package.json`), on relance le build, et l'étape d'installation des dépendances doit rester `CACHED`. Si elle se relance, l'ordre des instructions est encore à revoir.
- Adverse : `docker run --rm -it mon-image sh -c whoami` ne doit jamais afficher `root`. Si c'est le cas, n'importe quelle faille dans le code applicatif (une dépendance vulnérable, une injection) donne à un attaquant un accès root à l'intérieur du conteneur, avec accès à tout ce que ce conteneur peut voir ou monter : c'est la différence entre un incident contenu et une porte grande ouverte.

On pousse plus loin

Distroless et `scratch` : Alpine, c'est déjà petit, mais il reste un shell, un gestionnaire de paquets, quelques utilitaires système. Les images [distroless](#) de Google vont plus loin : elles ne contiennent que le runtime applicatif strictement nécessaire (par exemple le runtime Node.js) et rien d'autre, pas même un shell. Résultat : image encore plus petite, et surtout surface d'attaque réduite au minimum (pas de `sh` disponible pour un attaquant qui obtiendrait un accès). Encore plus radical, [scratch](#) est une image littéralement vide, réservée aux binaires compilés statiquement (typiquement du Go) qui n'ont besoin d'aucune dépendance système.

`docker build --no-cache` : force un rebuild complet en ignorant tout le cache de couches. Utile pour vérifier qu'un build reste reproductible depuis zéro, ou pour forcer la récupération d'une version plus récente d'une dépendance système que le cache masquerait sinon.

BuildKit et le cache de dépendances : le moteur de build par défaut de Docker aujourd'hui s'appelle [BuildKit](#). Il apporte, entre autres, les [cache mounts](#) : un cache persistant entre builds, indépendant du cache de couches classique. Par exemple, pour ne jamais retélécharger les paquets npm ou pip d'un build à l'autre, même quand le lock file change :

```
RUN --mount=type=cache,target=/root/.npm \  
  npm ci --omit=dev
```

Les builds multi-architecture : une image buildée sur un Mac Apple Silicon (architecture ARM64) ne tourne pas nécessairement sur un serveur x86_64 classique, et inversement. [docker buildx](#) permet de builder une même image pour plusieurs architectures en une seule commande, et de publier une image "manifeste" qui pointe automatiquement vers la bonne variante selon la machine qui la télécharge :

```
docker buildx build --platform linux/amd64,linux/arm64 -t mon-pseudo/mon-  
image:1.0 --push .
```

Le scan de vulnérabilités : `docker scan`, historique, est aujourd'hui remplacé par [Docker Scout](#), intégré nativement à Docker Desktop et au CLI :

```
docker scout cves mon-image:multistage
```

Autre option très utilisée en CI, [Trivy](#), un scanner open source qui couvre à la fois les vulnérabilités des paquets système, des dépendances applicatives, et les erreurs de configuration :

```
trivy image mon-image:multistage
```

docker history pour lire les couches : chaque couche d'une image reste inspectable, même si le fichier qu'elle contenait a été supprimé dans une couche suivante. C'est exactement pour ça qu'on ne doit jamais faire un `COPY .env .` suivi d'un `RUN rm .env` en pensant avoir nettoyé : le `.env` reste lisible dans la couche `COPY`.

```
docker history mon-image:multistage
```

Devrait afficher la liste des couches, leur taille individuelle, et la commande qui les a créées : de quoi repérer immédiatement une couche anormalement grosse, ou une commande qui n'aurait jamais dû laisser de trace dans l'image finale.

6 - Networks et Volumes : faire communiquer, faire persister

On sait maintenant construire une image et lancer un conteneur. Mais un conteneur tout seul, c'est une île : il ne parle à personne, et tout ce qu'il écrit disparaît dès qu'on le supprime. Dans un vrai projet, on a toujours plusieurs conteneurs (une API, une base de données, un cache) qui doivent se parler, et des données (comptes utilisateurs, uploads, logs) qui doivent survivre bien plus longtemps que le conteneur qui les a créées. C'est exactement ce que résolvent les networks (la communication) et les volumes (la persistance). Deux briques qu'on retrouve dans absolument tous les projets Docker en production.

L'essentiel des networks

C'est quoi un network Docker ?

Un [network Docker](#) est un réseau virtuel qui permet à des conteneurs de communiquer entre eux de manière isolée et sécurisée. C'est Docker qui gère toute la couche réseau à notre place, pour connecter des conteneurs (ou au contraire les isoler) sans toucher à la configuration réseau de la machine hôte. Sans network, des conteneurs sont comme des îles isolées qui ne peuvent pas se parler.

Dès qu'on lance plusieurs conteneurs qui doivent collaborer (API, base de données, cache), ils doivent pouvoir échanger des données. Le network Docker crée un pont de communication entre eux.

Pourquoi utiliser des networks Docker ?

Le problème, sans network personnalisé :

```
# Un conteneur qui joue le rôle d'un service web, il répond vraiment sur le
port 80
docker run -d --name example-web nginx

# Un second conteneur, celui qui va essayer de joindre le premier
docker run -dit --name example-client alpine sh
```

Depuis le client, on tente de joindre le service web par son nom :

```
docker exec example-client wget -qO- http://example-web
```

Devrait échouer avec `wget: bad address 'example-web'`. Les deux conteneurs tournent pourtant sur la même machine : ils atterrissent chacun sur le bridge par défaut, où aucun nom n'est résolu. Le premier ne sait tout simplement pas que le second existe.

La solution, avec un network :

```
# On repart propre
docker rm -f example-web example-client

# On crée un network dédié
docker network create example-network

# Et on relance les deux conteneurs dessus
docker run -d --name example-web --network example-network nginx
docker run -dit --name example-client --network example-network alpine sh
```

Maintenant, le client peut viser le service web directement par son nom, plus besoin de connaître une IP qui change à chaque redémarrage :

```
docker exec example-client wget -qO- http://example-web
```

Devrait afficher la page d'accueil HTML de nginx, celle qui commence par `<!DOCTYPE html>` et contient `Welcome to nginx!`. Le nom `example-web` a été résolu tout seul vers la bonne adresse IP : c'est le network custom qui fait ce travail.

Un peu de ménage avant de continuer :

```
docker rm -f example-web example-client
docker network rm example-network
```

Les 4 types de networks Docker

1. Bridge (par défaut)

- Le network le plus courant, celui qu'on utilise dans la grande majorité des projets
- Les conteneurs sur le même host peuvent communiquer entre eux
- Isolés du réseau externe (sauf si des ports sont exposés avec `-p`)
- Docker crée automatiquement un [bridge network](#) appelé "bridge" au démarrage du daemon

- Cas d'usage : isoler une stack applicative (API + base de données + cache) du reste de la machine, tout en laissant ses conteneurs se parler

2. Host

- Le conteneur partage directement le réseau de la machine hôte : aucune isolation réseau
- Performance maximale, pas de traduction de ports (NAT), le conteneur écoute directement sur les ports de l'hôte

NAT ? Network Address Translation. Le mécanisme qui réécrit les adresses et les ports au passage d'une frontière réseau : c'est lui qui fait qu'un `-p 5000:80` redirige le port 5000 de la machine vers le port 80 du conteneur. En mode host, cette réécriture disparaît, d'où le gain de performance.

- Cas d'usage : outils de mesure de performance réseau, ou services qui ont besoin d'un accès réseau bas niveau (reste rare en usage courant)
- Attention aux conflits de ports : si le conteneur écoute sur le port 3000, aucun autre processus de l'hôte ne peut l'utiliser en même temps

3. None

- Aucun réseau du tout, le conteneur est complètement isolé
- Cas d'usage : des tâches de calcul pur qui ne doivent avoir aucun accès réseau (traitement de fichiers, script de migration ponctuel, sandbox de sécurité)

4. Overlay

- Pour [Docker Swarm](#) ou Kubernetes (orchestration multi-serveurs)
- Permet à des conteneurs qui tournent sur des machines physiques différentes de communiquer comme s'ils étaient sur le même réseau local
- On le retrouvera dès qu'on passera à un orchestrateur : hors périmètre pour cette introduction, mais bon à garder en tête, on y revient plus tard dans le programme

Bridge par défaut, network custom : la vraie différence

Pourquoi prendre systématiquement le réflexe de créer un network custom, même pour deux malheureux conteneurs ?

Sur le bridge par défaut (celui que Docker crée tout seul, littéralement nommé "bridge"), les conteneurs se voient bien au niveau réseau, mais impossible de se joindre par leur nom : il faut connaître leur adresse IP interne, qui change à chaque redémarrage. Sur un bridge network personnalisé (celui qu'on crée soi-même avec `docker network create`), Docker active en plus un petit serveur DNS interne : chaque conteneur devient joignable par son nom, exactement comme un nom de domaine sur internet.

DNS ? Domain Name System. Le service qui traduit un nom lisible ("example-api-2") en adresse IP réelle. Docker en fait tourner une version miniature à l'intérieur de chaque network custom.

C'est LA raison pour laquelle on crée toujours un network custom en pratique : sans lui, il faut bricoler avec des adresses IP qui bougent à chaque redémarrage. Avec lui, `api` peut simplement viser `database` sans jamais se soucier de l'IP réelle.

Comment ça marche concrètement

DNS automatique. Quand plusieurs conteneurs rejoignent le même network custom, Docker active ce petit DNS interne dont on vient de parler. C'est exactement ce qu'on vient de vérifier avec `example-api` et `example-api-2` : aucun des deux n'a eu besoin de connaître l'adresse IP de l'autre, le nom a suffi.

Ce mécanisme s'applique à n'importe quelle paire de conteneurs qui doivent collaborer : une API et une base de données, un frontend et un backend, un service de cache et l'application qui s'en sert. Tant qu'ils sont sur le même network custom, le nom du conteneur fait office d'adresse stable, qui ne bouge jamais, même après un redémarrage.

Isolation réseau

Des conteneurs sur des networks différents ne peuvent PAS communiquer entre eux, sauf configuration explicite.

Network	Conteneurs présents	Qui peut parler à qui
Network A	api, db	api parle à db
Network B	front, worker	front parle à worker

Et `api` ne peut jamais rejoindre `front` ou `worker` : ils ne partagent aucun network en commun.

Commandes essentielles

Voilà la [référence complète des commandes docker network](#) :

```
# Lister les networks
docker network ls

# Créer un network
docker network create mon-network

# Inspecter un network (voir quels conteneurs sont dessus)
docker network inspect mon-network

# Connecter un conteneur existant à un network
docker network connect mon-network mon-conteneur

# Déconnecter un conteneur d'un network
docker network disconnect mon-network mon-conteneur

# Supprimer un network (aucun conteneur ne doit l'utiliser)
docker network rm mon-network

# Nettoyer tous les networks non utilisés
docker network prune
```

Exercice guidé : deux conteneurs qui se parlent par leur nom

Objectif : créer deux conteneurs qui communiquent via un network personnalisé, puis constater ce qui se passe quand un troisième en est exclu.

Étape 1, créer le network :

```
docker network create app-network
```

Étape 2, lancer un conteneur "serveur" :

```
docker run -dit --name serveur --network app-network alpine sh
```

Étape 3, lancer un conteneur "client" :

```
docker run -dit --name client --network app-network alpine sh
```

Alpine embarque déjà `ping` (fourni par BusyBox) : pas besoin d'installer quoi que ce soit pour l'étape suivante, ce qui évite les surprises d'un `apk add` qui échoue selon la version de l'image.

Étape 4, tester la communication :

```
docker exec client ping -c 3 serveur
```

Devrait afficher trois réponses, du style :

```
PING serveur (172.X.X.X): 56 data bytes
64 bytes from 172.X.X.X: seq=0 ttl=64 time=0.XXX ms
```

Étape 5, inspecter le network :

```
docker network inspect app-network
```

Devrait afficher les deux conteneurs listés, chacun avec son adresse IP interne.

Étape 6, tester l'isolation :

```
# On crée un second network
docker network create other-network

# On lance un conteneur sur cet autre network
docker run -dit --name isole --network other-network alpine sh

# On tente de pinguer depuis le client
docker exec client ping -c 3 isole
```

Devrait échouer avec un message du type `ping: bad address 'isole'` : les conteneurs sur des networks différents ne se voient pas, exactement comme prévu par la théorie.

Nettoyage :

```
docker stop serveur client isole
docker rm serveur client isole
docker network rm app-network other-network
```

L'essentiel des volumes

L'inconvénient des conteneurs Docker, c'est qu'ils sont tous isolés les uns des autres, et que si on les ferme, on perd toutes les données calculées dynamiquement à l'intérieur.

Comment rendre les données persistantes et partageables, indépendamment de la vie des conteneurs ? C'est le rôle d'un [volume Docker](#) : un espace de stockage géré par Docker, qui vit en dehors du système de fichiers éphémère du conteneur.

À quoi sert un volume Docker ?

- **Persistance des données.** Les conteneurs Docker sont éphémères : si on supprime ou recrée un conteneur, tout ce qui est stocké à l'intérieur (dans le système de fichiers du conteneur) est perdu. 🙌 Un volume permet de stocker des données de façon persistante, indépendamment du cycle de vie du conteneur. Par exemple, si une application écrit des fichiers dans `/stuff`, ces fichiers restent disponibles même si on détruit et recrée le conteneur.
- **Partage de données.** Un volume peut être monté dans plusieurs conteneurs en même temps. Cela permet de partager des fichiers ou des données entre plusieurs services ou applications qui tournent dans des conteneurs différents.
- **Séparation des données et de l'application.** Le volume est stocké sur l'hôte Docker, dans un emplacement géré par Docker (généralement sous `/var/lib/docker/volumes`). Cela sépare clairement les données de l'application, ce qui facilite la sauvegarde, la migration ou la gestion des données sans toucher à l'application elle-même.
- **Sécurité et portabilité.** Les volumes sont isolés du système de fichiers principal de l'hôte, ce qui réduit les risques de conflits ou de suppression accidentelle. Ils sont aussi plus portables que les montages de type "bind" (qui lient un dossier précis de l'hôte).

Voilà ce qu'il se passe, visuellement

Trois façons de monter du stockage

Le volume nommé n'est pas la seule option. Docker propose trois types de montage : le volume nommé, le [bind mount](#) qui pointe vers un dossier précis de l'hôte, et le [tmpfs mount](#) qui vit uniquement en mémoire.

Type de montage	Où sont stockées les données	Cas d'usage typique
Volume nommé (<code>docker volume create</code>)	Géré par Docker, dans <code>/var/lib/docker/volumes</code>	Persistance de données d'application : base de données, uploads, logs qui doivent survivre au conteneur

Type de montage	Où sont stockées les données	Cas d'usage typique
Bind mount (dossier de l'hôte)	N'importe où sur le système de fichiers de l'hôte, chemin choisi par nous	Développement en local : on édite le code sur son poste, le conteneur voit les changements immédiatement
tmpfs (mémoire)	Uniquement en RAM, jamais écrit sur disque	Données sensibles et temporaires (secrets en clair pendant un calcul), performance maximale, rien ne doit persister

-v ou --mount ? On tranche. `-v` (ou `--volume`) est la syntaxe historique, compacte mais ambiguë : `docker run -v shared-stuff:/stuff mon_conteneur`. Elle mélange dans la même écriture volume nommé et bind mount, et elle crée silencieusement un dossier manquant sur l'hôte si on se trompe de chemin. `--mount` est la syntaxe moderne, plus verbeuse mais explicite : `docker run --mount source=shared-stuff,target=/stuff mon_conteneur`. Elle nomme chaque paramètre (`source`, `target`, `type`) et refuse un chemin hôte inexistant plutôt que de le créer par erreur. Le réflexe à prendre : `--mount` par défaut, c'est plus clair à lire et moins piégeux. `-v` reste très courant en legacy, donc il faut savoir le lire.

Partager des données grâce aux volumes

Prenons un volume Docker nommé `shared-stuff` :

```
docker volume create shared-stuff
```

Une fois un conteneur lancé avec `docker run`, l'option `--mount` lie le volume créé au conteneur :

```
docker run --mount source=shared-stuff,target=/stuff mon_conteneur
```

Que faire d'un volume ensuite ?

- **Stocker des données d'application.** Le volume sert à y ranger tout ce qu'une application doit conserver : bases de données, fichiers de configuration, fichiers utilisateurs, logs, etc.
- **Sauvegarder ou restaurer.** On peut sauvegarder le contenu du volume (par exemple pour des backups réguliers) ou le restaurer sur un autre hôte ou environnement.
- **Partager entre plusieurs conteneurs.** Si plusieurs conteneurs ont besoin d'accéder aux mêmes données, le même volume se monte dans chacun d'eux.
- **Nettoyer ou supprimer.** Un volume devenu inutile se supprime avec `docker volume rm shared-stuff` (après avoir arrêté et supprimé les conteneurs qui l'utilisent).

Instant Cheat Sheet ! Manipuler le volume

Concrètement, voilà ce qu'on peut faire sur un volume, dans un terminal (référence complète : [docker volume](#)) :

- Voir le contenu d'un volume, depuis un shell dans le conteneur :

```
docker exec -it mon_conteneur sh
ls /stuff
```

- Copier un fichier depuis le volume vers la machine hôte :

```
docker cp mon_conteneur:/stuff/nom_du_fichier ./local_destination/
```

- Copier un fichier de la machine hôte vers le volume :

```
docker cp ./mon_fichier mon_conteneur:/stuff
```

- Lister tous les volumes Docker :

```
docker volume ls
```

- Inspecter un volume pour voir où il est stocké sur l'hôte :

```
docker volume inspect shared-stuff
```

- Supprimer le volume, après avoir supprimé tous les conteneurs qui l'utilisent :

```
docker rm mon_conteneur
docker volume rm shared-stuff
```

- Sauvegarder le contenu du volume :

```
docker run --rm \
  -v shared-stuff:/stuff \
  -v $(pwd) :/backup \
  alpine \
  tar czf /backup/backup-stuff.tar.gz -C /stuff .
```

- Restaurer une sauvegarde dans le volume :

```
docker run --rm \
  -v shared-stuff:/stuff \
  -v $(pwd) :/backup \
  alpine \
  tar xzf /backup/backup-stuff.tar.gz -C /stuff
```

Exercice guidé : un volume qui survit à ses conteneurs

Objectif : créer un volume partagé, écrire dedans depuis un premier conteneur, le lire depuis un second, puis vérifier que les données restent lisibles après suppression complète des deux conteneurs.

On commence par créer le volume, depuis le terminal de la machine :

```
# Si un volume du même nom traîne d'un essai précédent
docker volume rm todo-logs

# Le volume partagé
docker volume create todo-logs
```

On lance ensuite un premier conteneur qui monte ce volume sur son dossier `/data`. La syntaxe courte `-v` est utilisée ici parce que c'est celle qu'on croise partout, y compris dans les fichiers de configuration qu'on verra au chapitre suivant :

```
docker run -it --name todo-writer -v todo-logs:/data node:22-alpine sh
```

Le prompt qui s'affiche est maintenant celui de l'intérieur du conteneur. On y écrit deux fichiers, puis on ressort :

```
cd /data
echo "mon premier log" > first_log.log
echo "mon second log: $(date)" > second_log.log
exit
```

De retour sur la machine, on lance un second conteneur, totalement distinct du premier, qui monte le même volume :

```
docker run -it --name todo-reader -v todo-logs:/data node:22-alpine sh
```

Et depuis l'intérieur de ce second conteneur :

```
cd /data
cat first_log.log
cat second_log.log
exit
```

Devrait afficher le contenu des deux fichiers, prouvant que `todo-reader` voit ce que `todo-writer` a écrit, sans que les deux conteneurs aient jamais communiqué directement.

On supprime maintenant les deux conteneurs, purement et simplement :

```
docker rm todo-writer todo-reader
```

Et on recrée un tout nouveau conteneur, qui n'a jamais existé avant, en le pointant sur le même volume nommé :

```
docker run -it --name todo-checker -v todo-logs:/data node:22-alpine sh
```

Toujours depuis l'intérieur du conteneur :

```
cd /data
cat first_log.log
cat second_log.log
exit
```

Devrait afficher exactement le même contenu qu'avant, alors que `todo-writer` et `todo-reader` n'existent plus du tout. C'est la preuve concrète que le volume vit indépendamment des conteneurs qui le montent : il survit à leur suppression complète, seule sa propre suppression explicite (`docker volume rm`) ferait disparaître les données.

Au boulot sur la Todo API

Côté dépôt : deux missions, deux commits séparés. La persistance et le réseau sont deux changements de nature différente : ils ne racontent pas la même histoire dans l'historique, et un `git log` qui les distingue facilite grandement un futur revert ciblé.

Mission A, la persistance

Pour l'instant, les tâches de l'API ne vivent qu'en mémoire : un redémarrage du conteneur les efface toutes. On branche [PostgreSQL](#) (un système de gestion de base de données relationnelle open source) dans son propre conteneur, avec un volume nommé pour ses données, et l'API s'y connecte.

Pas de fichier de configuration tout prêt cette fois : Postgres se lance avec un `docker run` et les bonnes options, à la main, une par une (image officielle, variables d'environnement pour le nom de la base, l'utilisateur et le mot de passe, volume nommé monté sur le répertoire où Postgres stocke ses données). On sent vite le nombre d'options qui s'accumulent sur une seule ligne de commande : c'est exactement la douleur que le chapitre suivant va résoudre, et c'est voulu.

Détail qui va compter tout de suite après : sans `network custom`, ce nouveau conteneur Postgres atterrit sur le bridge par défaut, comme n'importe quel `docker run` sans `--network`. L'API ne pourra donc pas le joindre par un nom, seulement par son IP interne (visible avec `docker network inspect bridge` ou `docker inspect`). Cette limite est volontaire : elle rejoue très concrètement ce qu'on vient d'apprendre sur le bridge par défaut, avant que la Mission B ne la corrige pour de bon.

Ce qui doit être vrai à la sortie :

- un conteneur Postgres tourne, avec un nom stable et un volume nommé qui lui est dédié
- l'API sait se connecter à cette base (un client adapté à la stack du projet, par exemple `pg` côté Node.js) et y écrit vraiment les tâches créées, plus seulement en mémoire
- une tâche créée via l'API est encore là après un `docker stop` puis `docker start` du conteneur Postgres
- une tâche créée via l'API est encore là même après un `docker rm` du conteneur Postgres suivi d'un tout nouveau `docker run` pointant sur le même volume nommé

Vérification :

```
# On crée une tâche
curl -X POST http://localhost:3000/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"Tâche persistante","status":"todo"}'

# On arrête puis on relance le conteneur Postgres
docker stop <conteneur-postgres>
docker start <conteneur-postgres>

# La tâche doit toujours être là
curl http://localhost:3000/api/tasks
```

Dans le Journal de bord du README, notons la commande `docker run` complète utilisée pour lancer Postgres, ainsi que l'IP interne trouvée pour connecter l'API, et une phrase sur le nombre d'étapes manuelles que ça représente comparé à ce qu'on imagine possible avec un seul fichier.

Mission B, l'isolation réseau

L'API et la base tournent, mais la base reste jointe par une IP interne fragile, et pire, si elle publie son port avec `-p 5432:5432`, n'importe quel processus sur la machine peut s'y connecter. On corrige les deux problèmes d'un coup avec un network custom nommé.

Ce qui doit être vrai à la sortie :

- l'API et la base de données sont toutes les deux sur un même network Docker créé explicitement, et rejoignable par son nom
- l'API se connecte à la base par son nom de conteneur, plus jamais par une IP
- le conteneur Postgres ne publie plus aucun port vers la machine hôte : `-p 5432:5432` a disparu de sa commande de lancement
- toutes les routes CRUD de l'API continuent de répondre normalement

Vérification :

```
# Toutes les routes doivent continuer à répondre normalement
curl http://localhost:3000/api/tasks

# Depuis la machine hôte, un client Postgres ne doit plus rien trouver
psql -h localhost -U todo_user -d todo_db
```

psql ? Le client en ligne de commande officiel de PostgreSQL, pour envoyer des requêtes SQL directement à une base. Sans lui sous la main, une simple vérification de port avec `nc -zv localhost 5432` (l'utilitaire réseau `netcat`, présent sur la plupart des systèmes) fait aussi l'affaire : elle doit échouer.

Dans le Journal de bord du README, notons la commande `docker network create` utilisée, le nom exact retenu pour le network, et le résultat de la tentative de connexion depuis l'hôte après la fermeture du port : ce qui s'affiche, et pourquoi c'est le comportement attendu.

Ce qui casse si...

Cas nominal, les données survivent au conteneur. On supprime complètement le conteneur Postgres (`docker rm`, pas seulement un `stop`) puis on en relance un tout nouveau, pointé sur le même volume nommé. Les tâches créées avant la suppression doivent toujours apparaître dans l'API. Si elles ont disparu, le volume n'était probablement pas monté au bon chemin (celui où Postgres écrit réellement ses données).

Cas limite, supprimer un conteneur n'est pas supprimer un volume. On teste les deux séparément, pour de vrai. D'abord `docker rm` sur le conteneur Postgres suivi d'un nouveau `docker run` sur le même volume : les données sont toujours là. Puis, sur un volume de test jetable, `docker volume rm` : cette fois les données ont disparu pour de bon, aucun conteneur ne peut les récupérer. La différence entre les deux commandes est exactement la frontière entre "je change de conteneur" et "je perds mes données".

Cas adverse, la base meurt en pleine écriture. On lance un `docker kill` sur le conteneur Postgres pendant qu'une requête `POST /api/tasks` est en cours côté API. L'API doit renvoyer une erreur HTTP exploitable au client (un code 500 avec un message clair, par exemple), jamais planter silencieusement ni rester bloquée indéfiniment sans réponse.

Cas de vérification d'isolation, la base ne doit plus être joignable depuis l'hôte. Une fois la Mission B terminée (port retiré, network custom en place), un client Postgres lancé directement depuis la machine hôte, hors de tout conteneur, ne doit plus pouvoir joindre la base du tout : connexion refusée ou timeout. Si la connexion aboutit encore, le port `5432:5432` traîne probablement encore quelque part dans la commande de lancement.

Creusons

Le volume, une fois créé, il vit où exactement sur la machine ?

Sur Linux, un volume nommé vit littéralement dans

`/var/lib/docker/volumes/<nom_du_volume>/_data`. On peut aller y jeter un œil avec les droits root, mais on ne devrait jamais y toucher à la main : c'est le rôle de Docker, pas le nôtre. Sur Docker Desktop (macOS, Windows), ce chemin existe à l'intérieur de la petite machine virtuelle Linux que Docker Desktop fait tourner en coulisses, pas directement sur le système de fichiers local : `docker volume inspect` reste le bon réflexe pour retrouver un volume, plutôt que d'aller fouiller le disque.

Les pilotes de volume. Par défaut, tout volume Docker utilise le pilote `local` : les données restent sur le disque de la machine qui fait tourner le conteneur. D'autres [pilotes de volume](#) existent, sous forme de plugins, pour stocker les données ailleurs : sur un partage réseau NFS ou CIFS, ou directement sur un disque cloud (Amazon EBS, Azure Disk...). L'intérêt : un volume qui ne dépend plus d'une seule machine physique, utile dès qu'on veut qu'un conteneur redémarré sur un autre serveur retrouve ses données. C'est typiquement ce que gère un orchestrateur comme Kubernetes en coulisses.

Le piège classique du bind mount qui masque `node_modules`. Quand on monte tout un dossier de code avec un bind mount (`- ./code`), ça écrase aussi le `node_modules` installé pendant le build de l'image avec celui, souvent absent, du poste local. Résultat : le conteneur démarre, cherche ses dépendances, et ne les trouve plus. La parade classique consiste à ajouter un second montage qui protège spécifiquement `node_modules` du bind mount du dossier parent, pour qu'il garde la version installée dans l'image plutôt que celle (ou l'absence) du poste local.

Permissions et UID quand le conteneur tourne en non-root. Un conteneur qui déclare `USER` `node` (bonne pratique de sécurité vue au chapitre précédent) écrit ses fichiers dans le volume avec l'UID de cet utilisateur, disons 1000. Si l'utilisateur local n'a pas le même UID, les fichiers du volume appartiennent alors à un inconnu du point de vue de l'hôte, avec des `Permission denied` dès qu'on essaie de les éditer en local. Deux parades : aligner l'UID du conteneur sur celui de l'hôte au moment du build (`--build-arg UID=$(id -u)`), ou lancer le conteneur avec `--user $(id -u):$(id -g)` pour du développement en local.

`docker volume prune`, la commande qui peut coûter cher. Elle supprime tous les volumes qui ne sont rattachés à aucun conteneur, même arrêté. Le piège : un volume de backup jamais monté sur un conteneur qui tourne, ou un volume nommé dont le conteneur a disparu autrement qu'en `down`, tombe dans le filet. Avant de lancer un `prune`, un `docker volume ls` suivi d'un `docker volume inspect` sur les volumes suspects évite bien des mauvaises surprises.

Performance des bind mounts sur macOS et Windows. Sur ces deux systèmes, Docker Desktop fait tourner les conteneurs dans une machine virtuelle Linux : un bind mount doit donc traverser une couche de traduction de système de fichiers entre le disque macOS ou Windows et cette VM. Historiquement, ça se traduisait par des lenteurs sensibles sur les gros dossiers (un `node_modules` de plusieurs dizaines de milliers de fichiers, typiquement). Docker a beaucoup travaillé le sujet, notamment avec [VirtioFS sur macOS](#), qui réduit fortement le temps des opérations de système de fichiers par rapport à l'ancien mode gRPC FUSE. Sur Windows, le même type de gain vient de faire tourner un projet directement dans le système de fichiers natif de WSL2 plutôt que sur un chemin `/mnt/c/...`. Le réflexe à garder : sur un gros projet, un volume nommé pour `node_modules` est souvent plus rapide qu'un bind mount complet, en plus de régler le piège d'écrasement vu plus haut.

Sauvegarder et restaurer un volume, comme vu dans l'Instant Cheat Sheet plus haut, prend tout son sens dès qu'on migre une base d'un hôte à un autre : on sauvegarde le volume source dans une archive, on la transporte, on la restaure dans un volume neuf sur la machine cible. C'est une opération manuelle, mais c'est exactement le mécanisme que des outils de sauvegarde plus élaborés automatisent en coulisses.

Une ligne de commande pour créer un network, une autre pour créer un volume, plusieurs options `-e` à empiler pour Postgres, et il faut refaire tout ça dans le bon ordre à chaque redémarrage de la stack : ça commence à faire beaucoup à retenir et à retaper. C'est précisément le problème que Docker Compose règle, en décrivant toute cette stack dans un seul fichier. Et c'est justement la suite.

7 - Docker Compose et la configuration : toute la stack dans un fichier

Au chapitre précédent, l'API et Postgres tournaient déjà ensemble, mais à la main : un `docker network create`, un `docker volume create`, puis deux `docker run` à rallonge avec `--network`, `--mount`, une option `-e` par variable, dans le bon ordre, sans jamais se tromper de nom de conteneur. Ça fonctionne, mais ça devient vite intenable : à chaque redémarrage de la machine, il faut retaper les mêmes dix lignes, et le jour où un troisième service s'ajoute, ça grimpe encore. Ce qui manque, c'est un seul fichier qui décrit toute la stack, et une seule commande pour la faire tourner. C'est exactement le rôle de [Docker Compose](#).

L'essentiel de docker-compose

"C'est bien drôle de passer sa vie à taper des commandes dans le terminal, mais il n'y a pas plus rapide ?"

Eh bien si : au lieu de retenir plusieurs commandes complexes par projet, il existe un moyen de standardiser les règles dans un fichier nommé `docker-compose.yml`.

Docker Compose est l'outil officiel Docker qui décrit une stack multi-conteneurs (services, réseaux, volumes) dans un seul fichier YAML, puis la pilote avec une poignée de commandes au lieu d'une dizaine de `docker run`, `docker network create` et `docker volume create` tapés à la main.

YAML ? Un format de fichier texte fait pour être lisible par un humain, où la structure se lit à l'indentation plutôt qu'aux accolades. C'est le format de configuration le plus répandu dans l'écosystème DevOps : on le retrouvera à l'identique dans les pipelines et dans Kubernetes plus tard dans la semaine. Sa contrepartie : l'indentation compte vraiment, un espace de trop et le fichier devient invalide. [Voici un guide pas à pas pour créer et configurer un `docker-compose.yml`](#), utile pour revoir les mêmes notions posées différemment.

Un fichier `docker-compose.yml` repose sur trois grandes clés, toujours au même niveau : `services` (un bloc par conteneur qu'on veut faire tourner), `networks` (les réseaux custom sur lesquels ces services discutent) et `volumes` (les volumes nommés pour la persistance). Par défaut, Compose crée déjà un network qui relie tous les services d'un même fichier entre eux : `networks` ne devient obligatoire que si on veut en isoler certains ou en nommer un précisément.

Un `docker-compose.yml` peut se structurer de la sorte :

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    volumes:
      - ./code
  cache:
    image: redis:7
```

Redis ? [Redis](#) est un système de stockage clé-valeur en mémoire, utilisé ici pour illustrer la clé `image`. On le retrouvera comme cache dans un vrai contexte applicatif, mais ce n'est pas le sujet du jour.

Aparté : et la ligne `version: '3.8'` qu'on voit partout ? On croise encore beaucoup d'exemples en ligne qui commencent par `version: '3.8'` ou similaire. Cette clé est [désormais obsolète](#) depuis Compose V2 : l'outil sélectionne automatiquement le schéma le plus récent et ignore cette valeur, avec juste un avertissement si elle traîne encore. Autant la retirer directement de ses propres fichiers, c'est ce qu'on fait dans tous les exemples de ce cours.

Voici un exemple plus complet, cette fois avec une appli qui sert un blog et sa base de données, pour voir les mêmes clés à l'œuvre sur une stack qui fait vraiment quelque chose :

```
services:
  blog:
```



```

build: .
ports:
  - "8000:80"
environment:
  - APP_ENV=production
  - DB_HOST=mysql
depends_on:
  - mysql

mysql:
  image: mysql:8
  environment:
    MYSQL_DATABASE: blogdb
    MYSQL_USER: blog_user
    MYSQL_PASSWORD: blog_pass
  volumes:
    - mysql_data:/var/lib/mysql

volumes:
  mysql_data:

```

MySQL ? [MySQL](#) est un système de gestion de base de données relationnelle open source, comme Postgres mais avec ses propres outils et sa propre syntaxe d'administration.

Deux services seulement ici, et c'est déjà suffisant pour voir toutes les clés qui comptent. Un troisième service, un quatrième, ça ne change rien au principe : c'est toujours un bloc de plus sous `services`, avec les mêmes clés.

Quelques clés méritent qu'on s'y attarde, parce qu'elles cachent des pièges classiques.

build VS image. `image: mysql:8` télécharge une image déjà construite depuis un registre (Docker Hub ici). `build: .` construit l'image depuis un `Dockerfile` présent dans le répertoire indiqué, exactement comme un `docker build .` manuel. Un service a l'un ou l'autre, rarement les deux à la fois.

depends_on et son piège. Dans l'exemple, `blog` attend `mysql`. Sauf que `depends_on` attend seulement que le *conteneur* `mysql` ait démarré, pas que *MySQL à l'intérieur* soit prêt à accepter des connexions. Entre les deux, il peut se passer plusieurs secondes, largement de quoi faire planter `blog` au premier essai de connexion. La correction passe par un `healthcheck` couplé à `condition: service_healthy` :

```

services:
  mysql:
    image: mysql:8
    environment:
      MYSQL_DATABASE: blogdb
      MYSQL_USER: blog_user
      MYSQL_PASSWORD: blog_pass
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 5s
      timeout: 3s
      retries: 5

```

```
blog:
  build: .
  depends_on:
    mysql:
      condition: service_healthy
```

Healthcheck ? Une commande que Compose exécute régulièrement à l'intérieur du conteneur pour savoir s'il est vraiment opérationnel, pas juste démarré. Ici, `mysqladmin ping` ne répond que quand MySQL accepte des connexions. [Documentation officielle sur depends_on couplé aux healthchecks](#).

Sans ça, `blog` démarre trop tôt, plante, et selon la `restart` policy repart en boucle jusqu'à ce que MySQL finisse par être prêt : ça marche, mais avec des logs d'erreur qui font peur pour rien.

restart. On y touche peu en développement, mais c'est la politique qui décide si Docker relance un conteneur tout seul quand il s'arrête : `no` (jamais, la valeur par défaut), `always` (toujours, même après un reboot de la machine), `on-failure` (seulement si le conteneur s'est arrêté en erreur), `unless-stopped` (toujours, sauf si on l'a stoppé volontairement). En production, `unless-stopped` est le choix le plus courant.

ports VS expose. `ports: - "3000:3000"` publie le port sur la machine hôte : on peut taper `localhost:3000` depuis le navigateur. `expose: - "3000"` ouvre le port uniquement pour les autres conteneurs du même network, sans rien publier vers l'extérieur. Une base de données qui ne parle qu'à l'application n'a besoin que d'`expose`, jamais de `ports`.

L'avantage ultime, c'est qu'il ne reste plus que trois commandes principales à retenir :

```
docker compose build
```

C'est l'équivalent de la commande de build à rallonge qu'on tapait à la main.

```
docker compose up
```

C'est l'équivalent de tous les `docker run` qu'on enchaînait un par un.

```
docker compose down
```

Cette commande arrête et retire les conteneurs qui avaient été lancés avec `docker compose up`.

Ces trois-là suffisent pour démarrer, mais au quotidien on pioche vite dans une palette plus large :

- `docker compose up -d` : lance la stack en arrière-plan (detached), on récupère la main dans le terminal tout de suite. Devrait afficher quelque chose comme :

```
[+] Running 3/3
- Network myapp_default      Created
- Container myapp-db-1       Started
- Container myapp-app-1      Started
```

- `docker compose down` : arrête et supprime les conteneurs et le network par défaut, mais laisse les volumes intacts, les données restent.

- `docker compose down -v` : pareil, mais supprime aussi les volumes. **Attention** : c'est irréversible, toutes les données de la base partent avec, ce n'est jamais une commande à taper par réflexe.
- `docker compose logs -f` : suit les logs de tous les services en direct, un `tail -f` multiplexé et préfixé par nom de service.
- `docker compose ps` : liste les conteneurs de la stack avec leur état (`running`, `healthy`, `exited`...).
- `docker compose exec app sh` : ouvre un shell (ou n'importe quelle commande) dans un conteneur déjà démarré, pratique pour aller inspecter un service en cours de route sans le redémarrer.
- `docker compose build --no-cache` : reconstruit une image en ignorant totalement le cache de layers, utile quand une dépendance a changé mais que Docker croit à tort que rien n'a bougé.
- `docker compose up --build` : force la reconstruction des images avant de relancer, le combo qu'on tape le plus souvent juste après une modification de code.

L'essentiel de la configuration

Une stack qui se lance en une seule commande, c'est déjà énorme, sauf qu'elle reste pleine de valeurs codées en dur : mot de passe en clair dans le yaml, host, port... La brique qui manque, ce sont les variables d'environnement.

C'est quoi une variable d'environnement ?

Une variable d'environnement est une valeur (texte, nombre, URL) qu'on passe à un conteneur pour configurer son comportement sans modifier le code source.

Au lieu de coder en dur :

```
const dbHost = "localhost"; // En dur, mauvaise pratique
```

On utilise une variable d'environnement :

```
const dbHost = process.env.DB_HOST; // Flexible !
```

Pourquoi utiliser des variables d'environnement ?

- **Séparation du code et de la configuration.** Le même code tourne en développement, en staging et en production, juste en changeant les variables.
- **Sécurité.** Les secrets (mots de passe, clés API) ne se retrouvent jamais dans le code, donc pas de risque de commit accidentel sur GitHub.
- **Flexibilité.** On change la configuration sans rebuild de l'image, avec une configuration différente selon l'environnement.

Comment passer des variables d'environnement ?

Méthode 1 : via `docker run`

```
docker run -e DB_HOST=postgres -e DB_PORT=5432 mon-image
```

Méthode 2 : via `docker-compose.yml` (inline)

```
services:
  api:
    image: mon-api
    environment:
      - DB_HOST=postgres
      - DB_PORT=5432
      - NODE_ENV=production
```

Méthode 3 : via `docker-compose.yml` (syntaxe objet)

```
services:
  api:
    image: mon-api
    environment:
      DB_HOST: postgres
      DB_PORT: 5432
      NODE_ENV: production
```

Méthode 4 : via un fichier `.env`

```
# .env
DB_HOST=postgres
DB_PORT=5432
NODE_ENV=production
```

```
# docker-compose.yml
services:
  api:
    image: mon-api
    env_file:
      - .env
```

Différence entre `environment` et `env_file`

`environment` : variables explicites dans `docker-compose.yml`.

- Avantage : tout est visible dans le fichier.
- Inconvénient : on commit souvent le `docker-compose.yml` (risque de fuite).

`env_file` : variables dans un fichier séparé.

- Avantage : on peut ajouter `.env` au `.gitignore` (sécurité).
- Inconvénient : moins visible, un fichier de plus à maintenir.

Bonne pratique : utiliser les deux.

```
services:
  api:
    image: mon-api
    environment:
      NODE_ENV: production # Pas sensible, peut être commité
    env_file:
      - .env # Contient les secrets, dans .gitignore
```

Ordre de priorité des variables

Si une même variable est définie à plusieurs endroits, voici l'ordre, du plus prioritaire au moins prioritaire :

1. Variables passées avec `docker run -e`
2. Variables dans `environment:` du `docker-compose.yml`
3. Variables dans le fichier référencé par `env_file:`
4. Variables dans le Dockerfile (`ENV`)
5. Valeurs par défaut dans le code

Bon à savoir : la hiérarchie précise avec `docker compose`. La liste ci-dessus est la version simple. Dans le détail, [la documentation officielle](#) distingue une valeur explicite dans `environment:` (la plus forte), puis `env_file:`, puis, pour l'interpolation des `${VAR}` à l'intérieur même du fichier compose, [une variable exportée dans le shell l'importe sur la même variable définie dans le `.env` du répertoire](#) : le `.env` reste le filet de sécurité par défaut, le shell gagne toujours quand les deux existent.

Sécurité des secrets

Mauvaise pratique :

```
# docker-compose.yml (commité sur GitHub)
services:
  api:
    environment:
      DB_PASSWORD: super_secret_123 # Visible sur GitHub !
```

Bonne pratique :

```
# .env (dans .gitignore)
DB_PASSWORD=super_secret_123
```

```
# docker-compose.yml
services:
  api:
    env_file:
      - .env # Le fichier .env n'est jamais commité
```

```
# .gitignore
.env
```

Encore mieux, pour la production : Docker Secrets, qu'on voit un peu plus loin, ou des outils comme Vault, qui restent hors programme pour nous.

ARG et ENV : le piège du secret qui fuit au build

Un réflexe dangereux consiste à passer un secret via `--build-arg` en pensant qu'il ne laissera aucune trace puisqu'il "ne persiste pas dans le conteneur". C'est à moitié vrai, et la moitié qui reste vraie fait mal.

```
FROM node:18-alpine
ARG DB_PASSWORD
ENV DB_PASSWORD=$DB_PASSWORD
WORKDIR /app
COPY . .
CMD ["node", "server.js"]
```

```
docker build --build-arg DB_PASSWORD=secret123 -t demo-arg .
docker history --no-trunc demo-arg
```

Devrait afficher, quelque part dans la liste des layers, une ligne du type `ENV`

`DB_PASSWORD=secret123` en clair. N'importe qui a accès à l'image (un registre partagé, un `docker save`) peut lire ce secret avec un simple `docker history`, même sans jamais lancer le conteneur.

ARG vs ENV ? `ARG` déclare une variable disponible uniquement pendant le `docker build` : elle n'existe plus dans le conteneur lancé ensuite. `ENV` déclare une variable qui persiste dans l'image finale et sera visible dans tout conteneur démarré à partir de cette image.

[Référence Dockerfile officielle.](#)

Bonne pratique : aucun secret ne va jamais en `ARG` ni en `ENV` dans un Dockerfile. Les secrets voyagent par `env_file` au runtime, ou par un mécanisme de secrets dédié (on y revient dans quelques minutes), jamais bakés dans une image.

Utiliser les variables dans le code

Node.js :

```
const express = require('express');
const app = express();

const PORT = process.env.PORT || 3000;
const DB_HOST = process.env.DB_HOST || 'localhost';
const DB_PASSWORD = process.env.DB_PASSWORD;

if (!DB_PASSWORD) {
  throw new Error('DB_PASSWORD must be defined');
}

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

```
    console.log(`Connecting to database at ${DB_HOST}`);
  });
```

Variables avec des valeurs par défaut

```
services:
  api:
    environment:
      DB_HOST: ${DB_HOST:-postgres}  # Si DB_HOST n'existe pas, utilise
      "postgres"
      DB_PORT: ${DB_PORT:-5432}
```

Exercice guidé 1 : passer des variables d'environnement

Objectif : construire une petite API qui lit des variables d'environnement.

Étape 1 : le code de l'API

```
// server.js
const express = require('express');
const app = express();

const PORT = process.env.PORT || 3000;
const API_KEY = process.env.API_KEY;
const ENVIRONMENT = process.env.NODE_ENV || 'development';

app.get('/config', (req, res) => {
  res.json({
    port: PORT,
    hasApiKey: !!API_KEY,
    environment: ENVIRONMENT,
    message: `Running in ${ENVIRONMENT} mode`
  });
});

app.listen(PORT, () => {
  console.log(`Server started on port ${PORT}`);
  console.log(`Environment: ${ENVIRONMENT}`);
  console.log(`API Key provided: ${!!API_KEY}`);
});
```

Étape 2 : le Dockerfile

```
FROM node:18-alpine
WORKDIR /app
RUN npm init -y && npm install express
COPY server.js .
CMD ["node", "server.js"]
```

Étape 3 : tester avec docker run

```
# On build l'image
docker build -t env-test .
```

```
# On lance sans variables
docker run -p 3000:3000 env-test

# Dans les logs : mode development, pas d'API key

# On lance avec variables
docker run -p 3000:3000 \
  -e NODE_ENV=production \
  -e API_KEY=secret123 \
  -e PORT=3000 \
  env-test

# On teste
curl http://localhost:3000/config
```

Devrait afficher :

```
{"port":"3000","hasApiKey":true,"environment":"production","message":"Running in
production mode"}
```

Étape 4 : passer par docker-compose

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: production
      API_KEY: secret123
      PORT: 3000
```

```
docker compose up
curl http://localhost:3000/config
```

Devrait afficher exactement la même réponse JSON qu'à l'étape précédente.

Exercice guidé 2 : utiliser un fichier .env

Objectif : séparer les secrets dans un fichier `.env`.

Étape 1 : le fichier .env

```
# .env
NODE_ENV=production
API_KEY=super_secret_key_123
DB_HOST=postgres
DB_PORT=5432
DB_PASSWORD=db_secret_password
```

Étape 2 : le docker-compose.yml


```
services:
  api:
    build: .
    ports:
      - "${PORT:-3000}:3000" # Utilise PORT du .env ou 3000 par défaut
    env_file:
      - .env
```

Étape 3 : ajouter .env au .gitignore

```
echo ".env" >> .gitignore
```

Étape 4 : créer un .env.example (template pour l'équipe)

```
# .env.example (ce fichier EST commité)
NODE_ENV=development
API_KEY=your_api_key_here
DB_HOST=postgres
DB_PORT=5432
DB_PASSWORD=your_password_here
```

Étape 5 : tester

```
docker compose up
curl http://localhost:3000/config
```

Devrait afficher :

```
{"port":"3000","hasApiKey":true,"environment":"production","message":"Running in
production mode"}
```

Le workflow d'équipe qui en découle :

1. Chaque membre clone le repo
2. Copie `.env.example` vers `.env`
3. Remplit ses propres valeurs
4. Lance `docker compose up`

Cette semaine, on retrouvera exactement le même réflexe côté CI : les variables masquées de GitLab (ou GitHub Actions) jouent le rôle du `.env`, mais pour la pipeline plutôt que pour le poste local. On y revient plus tard.

À vous de jouer

Avant de lancer : un `.env` qui traîne déjà avec un mot de passe dedans et qui part dans un commit sur un dépôt public, ça se rattrape très mal une fois poussé. Le réflexe qui sauve : un `git status` avant chaque commit, pour vérifier qu'aucun fichier de secret ne s'est glissé dans la liste.

Mission A : la configuration sort du code

Jusqu'ici, les identifiants de la base sont écrits en dur : dans le code de connexion, et probablement dans les commandes `docker run` tapées au chapitre précédent. L'objectif : plus aucun identifiant de base en clair nulle part. On sépare la configuration du code avec [dotenv](#), une librairie qui charge des variables depuis un fichier `.env` dans `process.env`.

Avant (config en dur) :

```
const pool = new Pool({
  host: 'db',
  port: 5432,
  user: 'todo_user',
  password: 'todo_pass',
  database: 'todo_db',
});
```

Après (config externalisée) :

```
require('dotenv').config();

const pool = new Pool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
});
```

Le code ne change plus jamais entre dev, staging et prod : seule la config change. Deux fichiers à créer : un `.env` rempli avec les vraies valeurs, jamais commité, présent dans le `.gitignore`, et un `.env.example` avec les mêmes clés et des valeurs bidons, commité pour que n'importe qui puisse démarrer le projet. Comme dans l'exercice guidé plus haut, une variable obligatoire manquante doit faire planter le démarrage avec un message clair (`throw new Error(...)`), jamais un `undefined` silencieux qui casse trois couches plus loin.

Trois choses à vérifier avant d'enchaîner : le code de connexion ne contient plus aucune valeur en dur, un `git log` sur le repository ne fait apparaître aucun `.env` commité par erreur, et retirer une variable du `.env` fait échouer le démarrage avec un message exploitable plutôt qu'un crash mystérieux.

Mission B : toute la stack dans un fichier

Deux missions, deux commits : celle-ci mérite le sien, distinct de celui de la Mission A. Un changement de nature (la config, puis l'orchestration) ne partage jamais le même commit.

L'objectif : une seule commande pour démarrer l'API, Postgres et [Adminer](#) ensemble.

Adminer ? Une interface web d'administration de base de données, livrée en un seul fichier PHP, qui se branche sur Postgres (ou MySQL, SQLite...) pour parcourir et éditer les données sans taper de SQL à la main. On la garde pour toute la suite du cours : c'est le moyen le plus rapide de vérifier visuellement que la base contient bien ce qu'on croit.

Le `docker-compose.yml` à écrire doit satisfaire tout ça à la fois : les trois services démarrent sur le network custom du chapitre précédent, le volume nommé qui préserve les données Postgres est toujours là, la base n'est plus jamais publiée sur l'hôte (seulement joignable par les autres conteneurs du network), et zéro valeur en dur, tout vient du `.env` de la Mission A via l'interpolation `${...}`.

Devrait afficher quelque chose comme ceci au lancement (les noms exacts dépendent du nom de votre dossier de projet et de ceux choisis pour vos services) :

```
[+] Running 5/5
- Network todo-network      Created
- Volume "todo_pgdata"      Created
- Container todo-postgres-1  Healthy
- Container todo-adminer-1   Started
- Container todo-api-1       Started
```

Le repère qui compte, quels que soient les noms réels : cinq ressources créées (un network, un volume, trois conteneurs), et le conteneur Postgres qui passe par l'état `Healthy` avant que l'API démarre.

Pour vérifier que tout est bien branché :

```
docker compose ps
```

Les trois conteneurs doivent apparaître à l'état `running` (`healthy` pour Postgres). Un `curl` sur la route qui liste les tâches doit répondre avec du JSON, la preuve que l'API a réussi à se connecter à la base.

Reste Adminer : on ouvre le port qu'on a choisi de publier pour lui dans le navigateur, un formulaire de connexion apparaît. On y entre le nom du service Postgres tel que déclaré dans le `docker-compose.yml` comme serveur, et les valeurs de `DB_USER`, `DB_PASSWORD`, `DB_NAME` du `.env`. On doit atterrir sur la liste des tables de la base, la preuve visuelle que tout communique.

Dans le Journal de bord du README, on consigne ce qui a été mesuré et ce qui a cassé en chemin : le temps que le healthcheck Postgres a mis à passer `Healthy`, les éventuels ajustements de noms entre le network ou le volume créés à la main au chapitre précédent et ceux que Compose gère désormais tout seul, et toute variable qu'on a dû ajouter au `.env.example` en cours de route.

Trois scénarios à rejouer

Le cas nominal. La stack complète démarre avec `docker compose up -d`, les trois conteneurs passent `running`, et l'API répond correctement sur sa route de liste des tâches. C'est la mission B, terminée avec succès.

Le cas limite : une variable obligatoire manque. *Qu'est-ce qui se passe si on retire `DB_PASSWORD` du `.env` ?* On commente cette ligne, puis :

```
docker compose down
docker compose up -d
docker compose logs
```

On lit ce que les logs racontent, service par service, et on note dans le Journal de bord du README ce qu'on y a trouvé : quel service refuse de démarrer, ce que dit le message d'erreur, et l'état dans lequel `docker compose ps` laisse les conteneurs bloqués.

Ce qui se joue concrètement : Postgres refuse de démarrer proprement (ou démarre avec un mot de passe vide selon la version de l'image), et son healthcheck échoue en boucle. Comme l'API attend `condition: service_healthy` sur Postgres, elle ne démarre jamais : elle reste bloquée en `created`, jamais en `running`. Une variable manquante en configuration, ça ne casse jamais silencieusement avec Compose, ça bloque net le démarrage du service qui en dépend. On remet la ligne dans le `.env`, et on relance pour repartir sur une stack saine.

Le cas adverse : la base tombe pendant que tout tourne. Avec la stack up, on arrête le conteneur Postgres sans toucher au reste :

```
docker compose stop <nom-du-service-postgres>
curl http://localhost:3000/api/tasks
```

Que répond l'API à ce moment-là ? On observe la réponse et le code HTTP renvoyé, et on les note dans le Journal de bord : une erreur exploitable côté client est un succès, un crash silencieux du process ou un timeout interminable est un problème à corriger dans le code de connexion. On relance ensuite ce même service avec `docker compose start`, et on vérifie que l'API retrouve d'elle-même le chemin vers la base, ou si elle réclame un redémarrage complet : ça aussi, ça va dans le Journal de bord.

Approfondissons

Compose fait tourner la stack qu'on vient de construire très bien sur un poste de développement, mais un `docker-compose.yml` qui fonctionne en local n'est pas encore prêt pour la production. Quelques réflexes qui font la différence.

Un processus par conteneur. [Les bonnes pratiques Docker officielles](#) recommandent de découpler chaque préoccupation dans son propre conteneur (l'API d'un côté, la base de l'autre, l'admin dans un troisième) plutôt que d'empiler plusieurs processus dans un seul. Ça se scale, se redémarre et se debugge indépendamment.

Conteneurs sans état. Un conteneur doit pouvoir être détruit et recréé à tout moment sans perte de donnée : tout ce qui doit survivre va dans un service de stockage dédié (un volume nommé, une vraie base), jamais dans le système de fichiers du conteneur lui-même.

Logs sur la sortie standard. Une application qui écrit ses logs dans un fichier à l'intérieur du conteneur les perd au moindre redémarrage, et `docker compose logs` ne voit rien. La [méthodologie douze facteurs](#) recommande d'écrire sur `stdout` / `stderr` et de laisser l'environnement d'exécution router ces flux ailleurs.

Images épinglées par digest. `postgres:15` est un tag, et un tag peut pointer vers un contenu différent d'un jour à l'autre. En production, on préfère référencer l'image par son empreinte exacte, `postgres@sha256:...`, pour garantir que c'est strictement la même image partout. [Documentation officielle sur le pull par digest](#).

Healthcheck et politiques de redémarrage. On les a déjà vus plus haut : en production, `restart: unless-stopped` couplé à un `healthcheck` correctement calibré est la base pour qu'un conteneur qui plante reparte tout seul, sans qu'on ait besoin de s'en apercevoir en pleine nuit.

Limites de ressources. Sans limite, un conteneur qui fuit en mémoire peut affamer toute la machine. Avec Compose :

```
services:
  app:
    build: .
    deploy:
      resources:
        limits:
          cpus: "0.50"
          memory: 512M
```

Et avec un simple `docker run`, l'équivalent est `docker run --memory=512m --cpus=0.5 mon-image`. [Documentation officielle sur les limites de ressources](#).

`docker compose config` pour valider. Cette commande affiche le yaml final résolu et interpolé, toutes les `${VARIABLE}` remplacées par leur valeur réelle, sans rien lancer. Une variable manquante ou une indentation cassée saute immédiatement aux yeux, avant même de démarrer quoi que ce soit. C'est le premier réflexe à prendre quand un `docker compose up` se comporte bizarrement : avant de soupçonner un bug applicatif, on vérifie que le fichier résolu contient bien ce qu'on croit.

Fichiers d'override pour séparer dev et prod. Compose charge automatiquement un `docker-compose.override.yml` s'il existe à côté du fichier principal, et [fusionne les deux](#). On peut aussi enchaîner plusieurs `-f` explicitement : `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d`. Le fichier de base reste commun, et chaque environnement n'ajoute que ce qui change, ce qui évite d'entretenir deux fichiers complets en parallèle qui divergent avec le temps.

Profiles. Certains services n'ont de sens qu'en développement, Adminer en est un bon exemple : personne ne veut exposer une interface d'admin de base de données en production. Les [profiles](#) permettent de le déclarer explicitement :

```
# docker-compose.override.yml (chargé automatiquement en local)
services:
  adminer:
    profiles: ["dev"]
```

En local, `docker compose --profile dev up -d` lance tout, Adminer compris. En production, un simple `docker compose up -d` sans ce flag l'ignore complètement. Ça se combine bien avec les fichiers `-f` vus juste au-dessus : les profiles filtrent des services à l'intérieur d'un même fichier, les `-f` multiples combinent plusieurs fichiers, deux mécanismes complémentaires plutôt que concurrents.

Secrets Docker versus variables d'environnement. Une variable d'environnement reste visible dans `docker inspect`, dans la liste des process, parfois dans des logs de debug trop bavards. Les [secrets Docker](#) répondent à ce problème : la donnée sensible est montée comme un fichier en mémoire dans le conteneur, lisible uniquement par le processus qui en a besoin.

```
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password

secrets:
  db_password:
    file: ./secrets/db_password.txt
```

L'image officielle Postgres sait justement lire un mot de passe depuis un fichier via `POSTGRES_PASSWORD_FILE`, précisément pour permettre ce genre de montage.

Tout ça reste valable sur *une seule* machine. Compose ne sait pas répartir des conteneurs sur plusieurs serveurs, ne redémarre rien tout seul si la machine entière tombe, et ne fait aucune mise à l'échelle automatique face à un pic de trafic. Le jour où une seule machine ne suffit plus, c'est un tout autre outil qui prend le relais : on y arrive avec l'orchestration, plus tard cette semaine.

8 - Un second service, en Python

On a maintenant une stack qui tient debout toute seule : l'API Node.js dans son conteneur, Postgres avec son volume nommé et son network dédié, et tout ça piloté depuis un seul `docker-compose.yml` dont la configuration vit dans des variables d'environnement. Il est temps d'ajouter un second service, écrit dans un langage que la promo ne maîtrise pas forcément : le Python. On le fait maintenant parce qu'une stack réelle est presque toujours polyglotte (un service de données en Python à côté d'une API Node, c'est un motif qu'on croise partout en entreprise), et parce que brancher un service qu'on n'a pas écrit soi-même sans en déboguer la syntaxe, c'est exactement la compétence qu'on demande le jour où l'équipe data livre un microservice sans documentation.

L'essentiel du Dockerfile Python

La stack s'enrichit avec un deuxième service : une petite API [FastAPI](#), un framework web Python moderne qui génère automatiquement une documentation interactive de l'API à partir des type hints du code.

Type hints ? Des annotations facultatives, en Python, qui précisent le type attendu d'un paramètre ou d'une valeur de retour (`def get_stats() -> dict:`). Python ne les vérifie pas à l'exécution, mais des outils comme FastAPI les lisent pour valider les entrées et générer la documentation tout seuls. On la conteneurise tout de suite, avec le même réflexe que pour l'API Node.js : une image de base légère, un utilisateur non-root, un `HEALTHCHECK`.

```

# Base légère, équivalent Python de node:alpine niveau philosophie
# "slim" retire les paquets Debian non essentiels, sans les limitations plus
poussées d'Alpine
# (Alpine existe aussi pour Python, mais certaines libs C compilent moins bien
dessus)
FROM python:3.12-slim

# Ne pas écrire de fichiers .pyc pendant l'exécution : inutile dans un
conteneur
# éphémère, ça évite juste des I/O disque pour rien
ENV PYTHONDONTWRITEBYTECODE=1

# Force la sortie standard de Python à ne pas être bufferisée : sans ça, les
logs
# de l'appli peuvent rester coincés dans un buffer et ne jamais apparaître dans
# `docker logs` avant un moment (voire jamais si le process crash brutalement)
ENV PYTHONUNBUFFERED=1

WORKDIR /app

# Comme pour npm, on copie d'abord uniquement le fichier de dépendances
COPY requirements.txt .

# --no-cache-dir : pip ne garde pas de cache local de téléchargement, inutile
dans
# une image qu'on jette et reconstruit à chaque fois, ça fait juste grossir
l'image
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Contrairement à l'image node officielle, l'image python:slim ne fournit pas
# d'utilisateur non-root prêt à l'emploi : on le crée nous-même
RUN adduser --disabled-password --gecos "" appuser
USER appuser

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=3s \
  CMD python -c "import urllib.request;
  urllib.request.urlopen('http://localhost:8000/health')" || exit 1

# uvicorn : le serveur ASGI qui fait tourner l'app FastAPI. --host 0.0.0.0 est
# indispensable dans un conteneur : sur 127.0.0.1 (le défaut), l'app ne serait
# joignable que depuis l'intérieur du conteneur lui-même
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

ASGI ? Asynchronous Server Gateway Interface, l'équivalent asynchrone du WSGI historique de Python : la norme qui permet à un serveur comme [Uvicorn](#) de faire tourner une app web capable de gérer des requêtes en parallèle sans bloquer.

pip ? Le gestionnaire de paquets standard de Python, l'équivalent de `npm` côté Node.js : il lit un fichier de dépendances et les installe dans l'environnement courant.

Qu'est-ce qui change vraiment par rapport à Node.js ? Trois choses concrètes :

- **La gestion des dépendances** : `npm ci` lit un lock file (`package-lock.json`) et installe exactement les versions qui y sont épinglées. `pip install -r requirements.txt` installe ce que le fichier demande, mais sans lock file strict par défaut : deux installs à des dates différentes peuvent récupérer des versions mineures différentes. Pour un vrai pinning à la `npm ci`, on fige les versions dans `requirements.txt` lui-même (`fastapi==0.115.0` plutôt que `fastapi`).
- **Les wheels** : quand une dépendance Python contient du code C à compiler, pip peut soit trouver un *wheel* déjà compilé pour la plateforme cible (rapide), soit devoir compiler depuis les sources (lent, et ça demande des outils de compilation absents d'une image slim). C'est une source classique de build qui traîne en longueur ou qui plante : si ça arrive, le multi-stage build (un stage avec les outils de compilation, un stage final sans) est la même solution que côté Node.
- **Les logs qui disparaissent** : c'est le piège le plus sournois. Sans `PYTHONUNBUFFERED=1`, Python bufferise sa sortie standard quand elle n'est pas connectée à un vrai terminal, ce qui est exactement le cas dans un conteneur. Résultat : `docker logs mon-conteneur` peut rester vide pendant que l'appli tourne et logue en réalité très activement, juste retenue dans un buffer. Un classique de debug qui prend 20 minutes la première fois qu'on le rencontre, et 2 secondes toutes les fois suivantes une fois qu'on connaît le coupable.

À noter, et ça change la donne pour la suite : contrairement au reste de l'après-midi, ce Dockerfile-là est donné complet, et c'est assumé. La compétence visée ici n'est pas d'écrire un Dockerfile Python, c'est de brancher correctement un service qu'on ne maîtrise pas dans une stack qu'on maîtrise déjà.

On branche le service

Pense-bête commits : nouveau service, nouveau dossier, un commit qui l'introduit séparément du reste de la stack. Un relecteur doit pouvoir comprendre l'ajout sans dérouler tout l'historique.

La mission : `stats-api`, un service Python qui lit la même base PostgreSQL que l'API Node.js et expose le nombre de tâches par état. Le code applicatif est donné complet, il n'y a rien à écrire en Python : le travail, c'est de le faire tourner dans la stack.

Voici `stats_api/main.py`, à placer dans un nouveau dossier `stats_api/` à la racine du projet :

```
import os

import psycopg2
from fastapi import FastAPI, HTTPException

app = FastAPI()

# Ces deux constantes dépendent du schéma créé au chapitre 6 : à adapter au nom
# réel de la table de tâches et de sa colonne d'état avant de lancer le
service.
TABLE_NAME = "tasks"
STATUS_COLUMN = "status"

# Les états attendus, listés ici pour garantir des compteurs à zéro même quand
# un état n'a aucune ligne en base (table vide, ou état jamais encore utilisé).
KNOWN_STATUSES = ["todo", "in_progress", "done"]
```



```

def get_connection():
    # Ces noms de variables doivent être exactement ceux choisis au chapitre 7
    # pour l'API Node.js : les deux services lisent la même configuration, il
    # serait absurde qu'ils l'appellent différemment.
    return psycopg2.connect(
        host=os.environ["DB_HOST"],
        port=os.environ.get("DB_PORT", "5432"),
        dbname=os.environ["DB_NAME"],
        user=os.environ["DB_USER"],
        password=os.environ["DB_PASSWORD"],
        connect_timeout=3,
    )

@app.get("/health")
def health():
    # Volontairement indépendant de Postgres : un souci base ne doit pas faire
    # passer le conteneur lui-même pour mort aux yeux du HEALTHCHECK.
    return {"status": "ok"}

@app.get("/stats")
def get_stats():
    counts = {status: 0 for status in KNOWN_STATUSES}

    try:
        conn = get_connection()
    except psycopg2.OperationalError:
        # Jamais de stacktrace brut renvoyé au client : un code d'erreur clair
        # et un message que l'appelant peut logger tel quel.
        raise HTTPException(
            status_code=503,
            detail="stats-api ne parvient pas à joindre la base de données",
        )

    try:
        with conn.cursor() as cursor:
            # TABLE_NAME et STATUS_COLUMN sont des constantes internes, jamais
            # entrée utilisateur : l'interpolation ici ne rejoue pas le risque
            # d'injection SQL qu'on aurait avec un paramètre reçu du client.
            cursor.execute(
                f"SELECT {STATUS_COLUMN}, COUNT(*) FROM {TABLE_NAME} "
                f"GROUP BY {STATUS_COLUMN}"
            )
            for status, count in cursor.fetchall():
                counts[status] = count
    finally:
        conn.close()

    return counts

```

Et voici `stats_api/requirements.txt`, à placer juste à côté :

```
fastapi==0.115.0
uvicorn==0.32.0
psycopg2-binary==2.9.9
```

psycopg2-binary ? Le client PostgreSQL pour Python, en version précompilée : la variante `-binary` embarque directement les bibliothèques C nécessaires, ce qui évite exactement le piège de compilation évoqué juste au-dessus.

Deux points d'attention, et c'est là que se trouve tout le travail de réflexion de ce chapitre, puisqu'il n'y a pas une ligne de Python à écrire.

D'abord, `TABLE_NAME` et `STATUS_COLUMN`, en haut de `main.py`, doivent correspondre au nom exact de la table de tâches et de sa colonne d'état tels que chacun les a créés au chapitre 6. Sans schéma de référence commun, ces deux lignes sont probablement à modifier avant que le service ne réponde correctement.

Ensuite, les noms de variables d'environnement lus par `get_connection()` doivent correspondre à ceux retenus au chapitre 7 pour l'API Node.js. Si le `.env` du projet parle de `POSTGRES_USER` là où le code Python attend `DB_USER`, le service plantera au premier appel avec une erreur de clé manquante, ce qui est d'ailleurs le comportement souhaité : mieux vaut un démarrage qui échoue franchement qu'une connexion silencieusement bancale.

Reste à brancher `stats-api` dans la stack, avec le Dockerfile vu juste au-dessus placé dans `stats_api/Dockerfile` : un service supplémentaire dans le `docker-compose.yml`, construit depuis `./stats_api`, rejoignant le même network custom que `db` et `todo-api`, recevant sa configuration Postgres via les mêmes clés de variables d'environnement, et publiant son port vers l'hôte (8000 convient, puisque c'est celui qu'expose le Dockerfile).

Une fois la stack relancée, un appel à `http://localhost:8000/stats` devrait afficher quelque chose comme :

```
{"todo": 3, "in_progress": 1, "done": 5}
```

Et un appel à `http://localhost:8000/health` devrait afficher :

```
{"status": "ok"}
```

Pour vérifier que le service est bien sur le bon network et pas isolé dans son coin, `docker network inspect` sur le network custom doit lister `stats-api` à côté de `db` et `todo-api`.

Checklist de sortie

- **Cas nominal** : les compteurs renvoyés par `/stats` correspondent exactement au contenu réel de la table, vérifié par un `COUNT` manuel directement en base.
- **Cas limite** : la table est vide, `/stats` répond `200` avec des compteurs à zéro, jamais une erreur `500`.
- **Cas adverse** : la base est coupée (`docker stop db`), `/stats` répond avec un code d'erreur clair et un message exploitable, jamais un timeout silencieux ni un stacktrace Python brut renvoyé au client.

Pour les curieux

Un service de lecture séparé comme `stats-api`, qui n'écrit jamais dans la base et se contente d'agréger, c'est un motif d'architecture qu'on retrouve partout dès qu'un système grossit : on l'appelle parfois un *read replica* applicatif. L'intérêt : on peut le faire évoluer, le redémarrer, voire le faire planter sans jamais mettre en danger l'intégrité des données, puisqu'il ne fait jamais de `INSERT` ni de `UPDATE`. Certaines équipes vont jusqu'à donner à ce type de service un utilisateur PostgreSQL dédié avec des droits `SELECT` uniquement : même si le code applicatif a un bug, la base refuse toute écriture venant de ce service.

Le code donné ici ouvre une connexion à chaque appel à `/stats`, puis la ferme. Ça fonctionne, mais ça a un coût : établir une connexion TCP puis l'authentification PostgreSQL prend quelques millisecondes à chaque fois, et sous forte charge, c'est ce qui sature en premier. La solution classique en production, c'est un *pool* de connexions déjà ouvertes et réutilisées entre les requêtes, via une bibliothèque comme [psycopg2.pool](#) côté application, ou un proxy dédié comme [PgBouncer](#) devant la base.

Que se passe-t-il si `todo-api` et `stats-api` lisent et écrivent la même ligne au même instant ?

PostgreSQL gère ça nativement via son système de transactions et de verrous : une écriture en cours bloque ou fait attendre les autres écritures concurrentes sur la même ligne, selon le niveau d'isolation choisi. C'est une notion qui mériterait une session entière à elle seule, mais la retenir en une phrase suffit pour l'instant : deux services qui partagent une base ne sont jamais complètement indépendants, et c'est précisément pour ça que `stats-api`, qui ne fait que lire, est le cas le plus simple à raisonner.

9 - Publier l'image et redéployer depuis le registry

`stats-api` tourne maintenant à côté de `todo-api`, les deux construits depuis le code source local via `build` dans le compose. On passe à l'étape qui sépare vraiment le build du run : publier les deux images sur un registry, puis redéployer uniquement depuis ces images publiées. On le fait maintenant parce que c'est exactement cette étape qu'une pipeline automatisera dans les prochains jours : comprendre à la main ce que ça implique, c'est comprendre ce que la pipeline fera à notre place.

L'essentiel du registry

Un **registry** d'images, c'est un serveur qui stocke et distribue des images Docker, de la même façon qu'un `npm registry` stocke des paquets JavaScript. [Docker Hub](#) en est l'exemple public de référence : c'est là que vivent les images officielles `node`, `python`, `postgres` qu'on utilise depuis le début de la journée, et c'est là qu'on va pousser les nôtres.

La convention de nommage d'un tag suit le format `pseudo/projet:version` : le pseudo identifie l'auteur ou l'organisation, le nom du projet identifie l'image, et la version identifie précisément quelle construction de cette image on regarde. Le tag `latest`, lui, ne veut dire qu'une chose : "la dernière image poussée sans tag explicite". Sur un dépôt d'équipe où plusieurs personnes poussent des images, `latest` change de contenu à chaque push, silencieusement, sans qu'aucune version ne le distingue de la précédente. Un tag de version explicite (`1.0.0`, puis `1.0.1` ...) est le seul moyen de savoir avec certitude ce qui tourne réellement quelque part.

Une fois poussée, une image versionnée devient le véritable artefact livrable : ce n'est plus le code source qui compte pour déployer, c'est l'image, figée, testée, identique bit pour bit partout où elle tourne. C'est le principe même du build once, deploy everywhere qui structure le reste de la semaine, et c'est exactement le geste qu'une pipeline CI/CD reproduira automatiquement à chaque merge.

Direction le registry

Avant de publier : deux images à publier, donc potentiellement deux commits distincts : le Dockerfile de `stats-api` d'un côté, le fichier compose de production de l'autre. On évite le commit qui mélange les deux sujets.

La mission : se connecter au registry, taguer les deux images selon la convention vue plus haut, les pousser, puis écrire un second fichier compose, `docker-compose.prod.yml`, qui référence les images publiées au lieu de les construire depuis les sources. Le critère de réussite est net : dans un dossier vide ne contenant que ce fichier et un `.env`, la stack complète démarre sans qu'une seule ligne de code source ne soit présente sur la machine.

La connexion au registry, déjà croisée en début de journée :

```
docker login -u <mon_pseudo>
```

Le tag, qui associe un nom versionné à une image locale déjà construite :

```
docker tag todo-api:latest <mon_pseudo>/todo-api:1.0.0
docker tag stats-api:latest <mon_pseudo>/stats-api:1.0.0
```

Et le push, qui envoie l'image taguée vers le registry :

```
docker push <mon_pseudo>/todo-api:1.0.0
docker push <mon_pseudo>/stats-api:1.0.0
```

Reste à écrire `docker-compose.prod.yml` : chaque `build: .` du fichier compose d'origine y devient un `image: <mon_pseudo>/todo-api:1.0.0` (et son équivalent pour `stats-api`), en conservant le reste de la configuration, le network, les volumes, les variables d'environnement.

Ce qu'on vérifie avant de fermer

- **Nominal** : dans un dossier vide contenant uniquement `docker-compose.prod.yml` et un `.env`, `docker compose -f docker-compose.prod.yml up -d` démarre toute la stack sans accroc.
- **Limite** : une image poussée sans tag explicite se retrouve taguée `latest` par défaut. En équipe, le bon réflexe reste de toujours donner un tag de version explicite, pour éviter qu'un push écrase silencieusement ce que quelqu'un d'autre utilise.
- **Adverse** : `docker history` sur l'image publiée ne doit faire apparaître aucun secret, aucune valeur de `.env`, aucun jeton de connexion. Une couche supprimée par une instruction ultérieure du Dockerfile reste parfaitement lisible dans l'historique : si un `ARG` ou un `COPY` `.env` a fui dans une couche intermédiaire, la couche finale a beau ne plus l'exposer au runtime, l'information reste récupérable par quiconque a accès à l'image. C'est une faute grave, pas un détail cosmétique.

On va plus loin

Docker Hub public est parfait pour s'entraîner, mais une entreprise héberge presque toujours son propre registry privé : [GitLab Container Registry](#) intégré directement au dépôt de code, [AWS ECR](#) côté cloud Amazon, ou [Harbor](#) en solution open source auto-hébergée. Le principe reste identique, seule l'URL du registry change dans les commandes de login, tag et push.

L'authentification par mot de passe, pratique pour s'entraîner en solo, cède la place en entreprise à des jetons d'accès à durée de vie limitée et à portée restreinte (lecture seule, ou push limité à un projet précis) : un jeton qui fuit dans les logs d'une pipeline coûte beaucoup moins cher à révoquer qu'un mot de passe qui protège aussi d'autres services.

Un registry qui accumule des années de tags finit par peser lourd et coûter cher en stockage : les politiques de rétention automatisent la suppression des images trop anciennes ou jamais retirées d'un environnement, en général en gardant les dernières versions de chaque branche majeure et en purgeant le reste après une fenêtre de temps.

Publier une image ne garantit pas qu'elle vient bien de la pipeline qu'on croit : la signature d'images, avec des outils comme [cosign](#) dans l'écosystème [Sigstore](#), permet de prouver cryptographiquement qu'une image donnée a bien été construite par le processus attendu, et l'attestation de provenance va plus loin en documentant exactement quel commit, quel pipeline et quelles dépendances ont produit l'image. C'est un sujet qui prend de l'ampleur avec les exigences de sécurité de la chaîne logicielle.

Enfin, une équipe qui mélange des Mac Apple Silicon et des serveurs x86 en production se heurte vite à un problème d'architecture processeur : une image construite sur un Mac M1 ne tourne pas nécessairement sur un serveur x86 classique. Les images multi-architecture, construites avec [Docker Buildx](#), empaquettent plusieurs variantes sous un seul tag, et Docker choisit automatiquement la bonne au moment du `pull` selon la machine qui exécute la commande.

10 - Mesurer et optimiser

Les deux services tournent maintenant depuis des images publiées, sans une ligne de code source sur la machine : la séparation build / run est acquise. Reste à regarder ce qu'on a vraiment construit toute la journée, avec des chiffres plutôt qu'avec une impression. On le fait maintenant parce que c'est le moment naturel d'agrégation : les mesures dispersées entre les chapitres précédents se comparent enfin entre elles, et parce que ces réflexes de mesure sont exactement ceux qui comptent une fois qu'une pipeline construit ces mêmes images cinquante fois par jour.

L'essentiel des métriques d'image

Quatre métriques, quatre commandes, et surtout quatre lectures différentes de ce qu'elles révèlent.

La **taille**, avec `docker images`, donne le poids total de l'image telle qu'elle sera transférée à chaque `pull`. C'est la métrique la plus visible, mais elle ne dit rien de comment ce poids est réparti : deux images de même taille peuvent avoir des profils de cache radicalement différents.

Le **nombre de couches et leur poids individuel**, avec `docker history`, complète justement ce point aveugle : une image légère peut cacher une seule couche énorme (un `COPY .` mal placé qui embarque un dossier entier), et une image qui semble avoir beaucoup de couches n'est pas forcément lourde si chacune pèse quelques kilo-octets.

`docker history` ? Une commande qui liste, couche par couche, comment une image a été construite. Utile pour repérer une couche anormalement lourde ou un secret qui aurait fui dans une instruction `COPY`, comme vu au chapitre précédent.

Le **temps de build à froid contre à chaud** compare `time docker build --no-cache -t monimage .` (rien en cache, tout se reconstruit) à `time docker build -t monimage .` juste après (le cache Docker réutilise les couches identiques). L'écart entre les deux mesure directement l'efficacité de l'ordre des instructions du Dockerfile : plus l'écart est grand, mieux le cache est exploité. Attention cependant, cette mesure dépend de l'état de la machine qui build : un `docker system prune` avant la mesure à froid est nécessaire pour un résultat comparable dans le temps.

Le **temps écoulé entre le lancement d'un conteneur et sa première réponse HTTP réussie** capture autre chose encore : ce que l'utilisateur final ressent réellement au démarrage. Une image peut être minuscule et pourtant lente à répondre, si l'application exécute des migrations de base ou du travail d'initialisation lourd avant d'accepter sa première requête. Cette métrique-là est la seule des quatre qui parle vraiment de l'expérience de démarrage, ni la taille ni le nombre de couches ne la prédisent.

Le tableau de mesures

Toujours côté Git : ce tableau de mesures a sa place dans le README, versionné, pas juste dans un terminal qu'on referme. Un commit à chaque nouvelle série de mesures permet de comparer l'historique dans le temps, pas seulement l'état du jour.

Pour chacune des deux images produites aujourd'hui (`todo-api` et `stats-api`), un tableau à quatre colonnes prend forme :

Image	Taille	Couches (poids max)	Build froid / chaud	Temps 1re réponse HTTP
todo-api	...	...	...	...
stats-api	...	...	...	...

Une fois le tableau rempli une première fois, l'étape suivante consiste à optimiser puis remesurer : une image de base plus fine (`alpine` plutôt que l'image complète, quand c'est compatible avec les dépendances), un ordre de `COPY` qui maximise le cache (dépendances avant code applicatif), un multi-stage qui ne garde vraiment que le nécessaire dans le stage final, un `.dockerignore` complété si un fichier inattendu traîne dans l'image. Deux cibles chiffrées donnent un objectif concret à viser : `todo-api` sous les 150 Mo avec un build à chaud sous 5 secondes, `stats-api` sous les 180 Mo (l'image de base `python:slim` pèse mécaniquement plus lourd qu'une image `node:alpine`, viser trop bas dessus coûte plus cher que ça ne rapporte).

Si une optimisation fait grossir l'image au lieu de la réduire, c'est une régression, pas un détail à ignorer : le cas le plus fréquent, un `COPY --from=builder` qui récupère un dossier trop large (tout `/app` au lieu de juste `/app/dist` et `/app/node_modules`, ou tout `/app` côté Python au lieu du strict nécessaire). Une régression de ce type mérite une ligne expliquée dans le Journal de bord

du README : ce qui a été tenté, pourquoi ça n'a pas marché, ce qu'on retient pour la prochaine tentative.

Jusqu'où peut-on descendre en taille avant qu'une image devienne impossible à déboguer ? Il n'y a pas de réponse universelle : une image ultra-minimale sans shell ni outils de diagnostic complique furieusement un `docker exec` de dépannage en production. À chacun de pousser sa propre image jusqu'au point où la compression cesse d'être rentable, et de noter dans le Journal de bord du README l'endroit précis où ce compromis a été tranché.

Quel est le vrai coût d'un build à froid dans une pipeline qui tourne cinquante fois par jour ? Multiplier le temps de build à froid mesuré plus haut par cinquante donne un ordre de grandeur concret, en minutes de calcul cumulées sur une journée. Cette estimation, calculée à partir des propres chiffres du tableau et consignée dans le Journal de bord du README, donne une intuition chiffrée de pourquoi le cache de build devient un sujet d'optimisation sérieux dès qu'une pipeline tourne à cette fréquence.

Comment mesurer l'écart entre l'instant où le conteneur démarre et l'instant où le service est réellement prêt à répondre ? Le temps de démarrage du conteneur (`docker start` qui rend la main) et le temps de disponibilité réelle du service (première réponse HTTP à 200) ne sont pas la même chose, et l'écart entre les deux est justement ce que capture la quatrième métrique du tableau. Un petit script qui boucle sur `curl` toutes les quelques centaines de millisecondes juste après le `up -d`, jusqu'à obtenir un 200, donne une mesure directe de cet écart pour chacune des deux images : de quoi comparer, à consigner dans le Journal de bord du README, si `stats-api` met plus ou moins de temps que `todo-api` à devenir réellement joignable.

Avant de valider une optimisation

Trois vérifications avant de considérer une optimisation comme acquise, pas juste "ça a l'air plus petit" :

- le build à froid aboutit toujours sans erreur après un `docker system prune`, pas seulement en s'appuyant sur un cache local qui masquerait un problème,
- le temps de démarrage n'a pas régressé par rapport à la mesure précédente dans le tableau,
- l'image ne contient toujours aucun secret ni fichier de test embarqué par erreur, vérifiable avec `docker history` comme au chapitre précédent.

Le test qui rejoue toute la journée

Un dernier scénario, qui traverse plusieurs chapitres d'un coup. Dans un dossier tout neuf, avec uniquement `docker-compose.prod.yml` et un `.env` reconstruit à partir de `.env.example`, la stack complète démarre depuis les images publiées. Puis, dans l'ordre : un `POST` avec un champ obligatoire manquant, qui doit être refusé proprement plutôt que planter le service ; une vérification que `localhost:5432` reste injoignable depuis la machine hôte, comme configuré au chapitre 6 ; une interrogation de `/stats`, comparée au contenu réel de la table par un `COUNT` manuel ; et pour finir, le conteneur `db` tué en pleine charge, pour observer si `todo-api` et `stats-api` dégradent proprement (erreurs claires, codes HTTP explicites) ou s'écroulent silencieusement.

Tout ce qui casse à cette étape se note dans le Journal de bord du README, avec ce qui a été observé et pourquoi. Ce test-là n'a pas vraiment de fin propre : on peut l'enrichir encore, ajouter un troisième service qui dépend des deux premiers, simuler une latence réseau artificielle entre les conteneurs, ou mesurer combien de temps il faut à la stack entière pour redevenir saine après que `db` redémarre. Il y a toujours une couche de robustesse supplémentaire à aller chercher.

Récapitulatif Jour 1

Ce qu'on a appris aujourd'hui

1. **DevOps et ses 3 piliers** : Culture, Automation, Measurement, le cycle Plan vers Monitor, et les 4 métriques qui disent si ça marche
2. **Ce qu'est vraiment un conteneur** : un processus Linux isolé par des namespaces et bridé par des cgroups, pas une machine virtuelle. On en a fabriqué un à la main pour le prouver
3. **L'architecture Docker et son vocabulaire** : client, daemon, registry, et les six briques Container, Image, Registry, Volume, Network, Dockerfile
4. **Le cycle de vie d'un conteneur** : `pull`, `create`, `start`, `run`, `stop`, `rm`, `rmi`, et pourquoi supprimer un conteneur ne supprime pas son image
5. **Notre première image, construite ensemble** : une app Node.js, un Dockerfile minimal, et les trois ports qui doivent raconter la même histoire
6. **Le Dockerfile de production** : layers et cache de build, `.dockerignore`, utilisateur non-root, multi-stage. Cette fois transposé sur votre propre projet, pas recopié
7. **Les Networks** : deux conteneurs ne se voient pas par défaut, un network custom leur donne une résolution par nom, et deux networks séparés isolent vraiment
8. **Les Volumes** : rendre les données persistantes et partageables, indépendamment de la vie des conteneurs
9. **Docker Compose** : toute la stack dans un fichier, configurée par variables d'environnement, avec les secrets hors du dépôt
10. **Une stack polyglotte** : un service Python branché sur la même base qu'une API Node.js, sans que l'un sache quoi que ce soit du langage de l'autre
11. **Publier une image** : `tag`, `push`, et le fait de pouvoir redéployer sa stack sans le code source. C'est exactement ce qu'automatisera notre pipeline plus tard dans la semaine
12. **Mesurer avant d'optimiser** : taille, couches, build à froid contre build à chaud. Une optimisation qu'on ne mesure pas est une croyance, pas un résultat

Points d'attention pour demain

- **Votre Todo API doit être dockerisée et fonctionnelle** : CRUD de tâches, base Postgres avec volume de persistance, configuration par variables d'environnement, network custom, et au moins une image publiée sur votre registry
- **Le journal de bord du README compte autant que le code** : si des mesures ou des pannes sont restées dans un terminal fermé, elles sont perdues. Un quart d'heure pour le remplir proprement, c'est du temps rentabilisé

- **Un dernier réflexe avant de fermer le portable** : le `.env` n'est pas commité, le `.env.example` l'est. C'est ce qui vous évitera de fuiter un mot de passe le jour où le dépôt devient public
- **Ce qui n'a pas été fini reste ouvert** : personne n'est censé avoir bouclé les dix chapitres, ils sont calibrés pour ça. Ce qui compte, c'est la trace de ce que vous avez cherché
- **Demain on approfondit Docker** : diagnostic et debug quand ça casse, optimisation d'images, healthchecks, et on commence à regarder ce qui se passe quand une seule machine ne suffit plus